

# Fast Verification of Strong Database Isolation

Zhiheng Cai  
Tsinghua University

Si Liu  
ETH Zurich

Hengfeng Wei  
Nanjing University

Yuxing Chen  
Tencent Inc.

Anqun Pan  
Tencent Inc.

## Abstract

Strong isolation guarantees, such as serializability and snapshot isolation, are essential for maintaining data consistency and integrity in modern databases. Verifying whether a database upholds its claimed guarantees is increasingly critical, as these guarantees form a contract between the vendor and its users. However, this task is challenging, particularly in black-box settings, where only observable system behavior is available and often involves uncertain dependencies between transactions.

In this paper, we present VERISTRONG, a fast verifier for strong database isolation. At its core is a novel formalism called hyperpolygraphs, which compactly captures both certain and uncertain transactional dependencies in database executions. Leveraging this formalism, we develop sound and complete encodings for verifying both serializability and snapshot isolation. To achieve high efficiency, VERISTRONG tailors SMT solving to the characteristics of database workloads, in contrast to prior general-purpose approaches. Our extensive evaluation across diverse benchmarks shows that VERISTRONG not only significantly outperforms state-of-the-art verifiers on the workloads they support, but also scales to large, general workloads beyond their reach, while maintaining high accuracy in detecting isolation anomalies.

## PVLDB Reference Format:

Zhiheng Cai, Si Liu, Hengfeng Wei, Yuxing Chen, and Anqun Pan. Fast Verification of Strong Database Isolation. PVLDB, 1(1): XXX-XXX, 2026. doi:XX.XX/XXX.XX

## PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://github.com/CzxingcHen/VeriStrong>.

## 1 Introduction

Strong isolation levels or guarantees—the I in ACID [35]—such as SERIALIZABILITY and SNAPSHOT ISOLATION, are essential for ensuring data consistency and integrity in modern databases. These guarantees prevent subtle concurrency anomalies, such as lost updates, that can lead to incorrect application behavior. Once a database is deployed, its promised isolation guarantees become a binding contract between the vendor and its users. This raises a critical question: *How can we be sure that the database upholds its promise?*

Answering this question is nontrivial. Modern databases have large codebases that are often inaccessible or too complex to reason about, even when available. This makes full verification of isolation

guarantees practically infeasible. Black-box verification [5, 32] offers an effective alternative: a verifier collects execution histories of database transactions as an *external observer* and checks whether these histories satisfy the isolation level in question.

However, verifying database histories remains challenging. The verification problem for strong isolation levels has been proven to be NP-hard, even for verifying a single history [5]. This complexity arises from the fact that, as an external observer of a black-box database, a verifier must *infer* the internal execution order of transactions, e.g., determining which transaction wrote an earlier version and which one wrote a later one. This challenge is further compounded by practical constraints: given limited time and memory, only a finite number of “guesses” can be made by the verifier, and thus only a finite number of histories can be verified. Yet, examining more histories is highly desirable, either to increase the likelihood of uncovering anomalies or to strengthen confidence in their absence.

Recent years have seen significant progress in accelerating the verification of strong isolation guarantees [5, 14, 18, 32, 34, 37]. Representative verifiers include Cobra [32] for SERIALIZABILITY, PolySI [14] and Viper [37] for SNAPSHOT ISOLATION, and Elle [18] for both (a detailed discussion is provided in Section 8). Despite these advances, verification efficiency remains a key limitation. Many verifiers have attempted to reduce verification overhead by leveraging advanced SMT (Satisfiability Modulo Theories) techniques [4]. However, general-purpose SMT solvers are prone to missing optimization opportunities specific to database verification.

Beyond this, two additional challenges make efficient verification even harder. First, all existing verifiers assume that transaction histories contain *unique write values*, i.e., each read can be deterministically matched to a single write. This greatly simplifies the task of inferring the internal execution order and serves as a key enabler of Elle’s design in particular. Yet this assumption does not hold in many real-world applications, where different transactions often write the same value. For instance, in an e-commerce platform, multiple users may submit identical orders for the same quantity of an item; in social media, thousands of users may “like” the same post. Likewise, standard database benchmarks that closely reflect real-world workloads, such as Twitter [17] and RUBiS [31], include *duplicate write values* by default. In fact, in our black-box setting, a (cloud) database is unaware that it is under test; we act purely as an external monitor, collecting the execution results of real workloads processed by the system. Hence, supporting such general workloads is highly desirable and essential for practical applicability.

Moreover, achieving both soundness (no false alarms) and completeness (no missed bugs) without compromising efficiency remains challenging in verifying strong isolation. In particular, identifying all anomalies in a history is computationally expensive, yet critical: overlooking even a single anomaly can require significant effort to rediscover. This challenge is further exacerbated by recent findings [23, 26], which reveal that certain anomalies arise *only* when duplicate write values are present—precisely the scenarios

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing [info@vldb.org](mailto:info@vldb.org). Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. 1, No. 1 ISSN 2150-8097.  
doi:XX.XX/XXX.XX

exemplified in real-world workloads above. Hence, relying on existing verifiers with unique write values risks both false negatives and false positives, undermining the reliability of verification results.

**Our Solution.** We present a novel black-box verifier, VERISTRONG, that checks database histories against the two most widely used strong isolation guarantees, i.e., SERIALIZABILITY and SNAPSHOT ISOLATION. VERISTRONG tackles the aforementioned challenges through an end-to-end verification pipeline. This includes (i) representing and encoding general database histories, (ii) establishing a sound and complete correspondence between the encoding and the verification algorithm, and (iii) optimizing checking efficiency using SMT techniques tailored for strong isolation verification.

At the core of our solution is a new formalism called *hyper-polygraphs*, which captures both certain transactional dependencies (e.g., session order) and uncertain ones (e.g., version order). In black-box settings, uncertainties can also arise from ambiguous read-from relations caused by duplicate write values—an aspect that existing formalisms, such as polygraphs [28, 35] and their variants [14, 32, 37], cannot adequately express. Hyper-polygraphs offer two key advantages. First, they provide a compact and expressive means of encoding transactional dependencies, making them particularly amenable to SMT-based verification. Second, they are generic, capable of representing a wide range of dependency-based isolation theories [1, 5, 7]. In this paper, we focus on Adya’s theory [1], with another application discussed in Section 9.

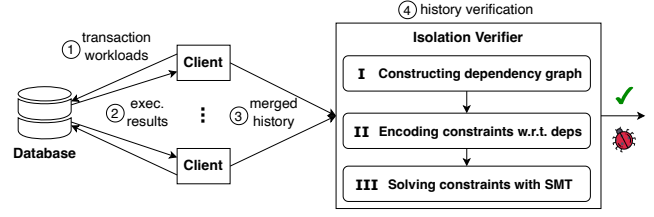
Building on this formalism, we develop sound and complete encodings for verifying both strong isolation guarantees, providing a rigorous theoretical foundation for reliable SMT-based verification. This enables our verifier to accurately (re)discover isolation anomalies in production databases (e.g., MariaDB), particularly those arising from duplicate write values that prior tools fail to handle.

To make verification fast, our key insight is that leveraging workload-specific knowledge can greatly boost SMT solving efficiency. To this end, we propose two domain-specific optimizations. First, we offload part of the solver’s effort to a lightweight preprocessing phase by proactively resolving small conflict patterns. This reduces expensive backtracking during solving by eliminating many infeasible search paths early and strikes a practical balance between preprocessing and solving costs, improving overall performance.

Second, we design a polarity picking strategy—i.e., deciding whether to explore a dependency—guided by a dynamically maintained pseudo-topological order during SMT solving. Unlike general-purpose solvers that make such decisions “blindly”, we leverage known partial orders, such as session order and certain read-from edges, to align these decisions with the likely transaction schedule enforced by the database. This alignment steers the solver away from conflict-prone search paths, thereby enhancing efficiency.

**Contributions.** Overall, we make the following contributions.

- (1) We introduce hyper-polygraphs, a new formalism that compactly characterizes isolation levels, including SERIALIZABILITY and SNAPSHOT ISOLATION, over general database histories.
- (2) We formally establish the soundness and completeness of our new characterizations, grounded in Adya’s theory, providing a theoretical foundation for reliable SMT-based verification.



**Figure 1: Workflow for verifying isolation guarantees in deployed databases. Steps II and III are specific to SMT-based approaches (see also Sections 4 and 6).**

- (3) We develop novel isolation verification algorithms by tailoring SMT solving to the characteristics of database workloads, incorporating optimizations that reduce solver overhead.
- (4) We implement our approach in VERISTRONG and assess it extensively on a variety of benchmarks. Results show that VERISTRONG not only substantially outperforms state-of-the-art verifiers on the workloads they support, but also scales to large, general workloads beyond their reach, while maintaining high accuracy in detecting isolation anomalies.

This work complements recent advances in isolation verification along two dimensions. First, it complements prior efforts [20, 25] on black-box verification of weaker levels like READ COMMITTED, which are known to be less complex to verify [5]. Second, it targets deployed databases, complementing recent work on verifying isolation in their designs [11, 22] and implementations [24].

We focus on SERIALIZABILITY as the running example throughout the paper, unless noted otherwise. We defer to the Appendix [6] how our approach extends to verifying SNAPSHOT ISOLATION.

## 2 Background

Black-box verification of database isolation guarantees involves four key steps, as illustrated in Figure 1. In Step ①, clients issue transactional requests to the database. In Step ②, each client records the corresponding execution results, including the values read or written and the status of each transaction (either committed or aborted). Next, in Step ③, the logs from all clients are merged into a single history, which is then passed to an isolation verifier. Finally, in Step ④, the verifier analyzes the history to determine whether it satisfies the specified isolation guarantee.

How does the verifier make this decision? It constructs a dependency graph based on, e.g., Adya’s theory [1] that is the *de facto* formalization of isolation guarantees. The verifier then searches the graph for cycles, which indicate violations of guarantees such as SERIALIZABILITY. Next, we recall the formal definitions underlying the history verification steps I–III from [14].

**Relations.** A binary relation  $R$  over a given set  $A$  is a subset of  $A \times A$ , i.e.,  $R \subseteq A \times A$ . For  $a, b \in A$ , we use  $(a, b) \in R$  and  $a \xrightarrow{R} b$  interchangeably. We use  $R^?$  and  $R^+$  to denote the reflexive closure and the transitive closure of  $R$ , respectively. A relation  $R \subseteq A \times A$  is *acyclic* if  $R^+ \cap I_A = \emptyset$ , where  $I_A \triangleq \{(a, a) \mid a \in A\}$  is the identity relation on  $A$ . Given two binary relations  $R$  and  $S$  over the set  $A$ , we define their composition as  $R \circ S = \{(a, c) \mid \exists b \in A. a \xrightarrow{R} b \xrightarrow{S} c\}$ . A strict partial order is an irreflexive and transitive relation. A strict total order is a relation that is a strict partial order and total.

**Transactions.** We consider a distributed key-value store that manages a set of keys  $\text{Key} = \{x, y, z, \dots\}$ ; each key is associated with a value from a set  $\text{Val}$ . The set of operations, denoted by  $\text{Op}$ , consists of both read and write operations:  $\text{Op} = \{R_i(x, v), W_i(x, v) \mid i \in \text{OpId}, x \in \text{Key}, v \in \text{Val}\}$ , where  $\text{OpId}$  is the set of operation identifiers. For simplicity, operation identifiers may be omitted. We use  $\_$  to represent an irrelevant value in an operation, e.g.,  $R(x, \_)$ , which is implicitly existentially quantified.

Clients interact with the key-value store by issuing transactions.

**Definition 2.1.** A *transaction* is a pair  $(O, \text{po})$ , where  $O \subseteq \text{Op}$  is a finite, non-empty set of operations;  $\text{po} \subseteq O \times O$  is a strict total order, referred to as the *program order*, which indicates the execution order of operations within the transaction.

For a transaction  $T$ , we write  $T \vdash W(x, v)$  if  $T$  writes to the key  $x$  and  $v$  is the last written value, and  $T \vdash R(x, v)$  if  $T$  reads from  $x$  before writing to it, and  $v$  is the value returned by the first such read. We also define  $\text{WriteTx}_x = \{T \mid T \vdash W(x, \_)\}$  to denote the set of transactions that write to  $x$ .

**Histories.** Transactions are grouped into client *sessions*, where each session consists of a sequence of transactions. We use *histories* to record the client-visible outcomes of these transactions.

**Definition 2.2.** A *history* is a pair  $\mathcal{H} = (\mathcal{T}, \text{SO})$ , where  $\mathcal{T}$  is a set of transactions with disjoint sets of operations;  $\text{SO} \subseteq \mathcal{T} \times \mathcal{T}$  is the *session order*, which is a union of strict total orders over disjoint sets of  $\mathcal{T}$ , each corresponding to a distinct client session.

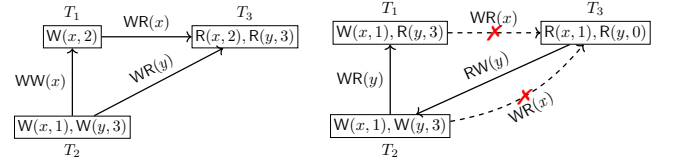
In line with existing formal models [1, 5, 7], we consider only committed transactions in a history; aborted transactions are handled separately. We also assume that every history contains a special transaction  $T_\perp$  that writes the initial values of all keys.<sup>1</sup> This transaction precedes all the other transactions in  $\text{SO}$  across sessions. Note that existing work commonly assumes UniqueValue histories, where each write to the same key assigns a distinct value, whereas real-world scenarios often involve DuplicateValue writes.

**Dependency Graphs.** A dependency graph extends a history by introducing three types of relations (or edges), WR, WW, and RW, each capturing a different kind of dependency between transactions. These relations underlies the formalization of isolation guarantees in the style of Adya [1, 7]. The WR relation connects a transaction that reads a value to the transaction that wrote it. The WW relation defines a strict total order, also referred to as the *version order* [1], among transactions that write to the same key. The RW relation is derived from WR and WW, associating a transaction that reads a value to the one that overwrites it based on the version order.

**Definition 2.3.** A *dependency graph* is a tuple  $\mathcal{G} = (\mathcal{T}, \text{SO}, \text{WR}, \text{WW}, \text{RW})$ , where  $(\mathcal{T}, \text{SO})$  is a history and

- $\text{WR} : \text{Key} \rightarrow 2^{\mathcal{T} \times \mathcal{T}}$  is such that
  - $\forall x \in \text{Key}. \forall S \in \mathcal{T}. S \vdash R(x, \_) \implies \exists! T \in \mathcal{T}. T \xrightarrow{\text{WR}(x)} S$
  - $\forall x \in \text{Key}. \forall T, S \in \mathcal{T}. T \xrightarrow{\text{WR}(x)} S \implies T \neq S \wedge \exists v \in \text{Val}. T \vdash W(x, v) \wedge S \vdash R(x, v)$ .
- $\text{WW} : \text{Key} \rightarrow 2^{\mathcal{T} \times \mathcal{T}}$  is such that  $\forall x \in \text{Key}$ ,  $\text{WW}(x)$  is a strict total order on  $\text{WriteTx}_x$ .

<sup>1</sup>In practice, we use multiple short, write-only transactions to initially populate the database. These transactions can be viewed as a single, logical write-only transaction.



(a) A serializable UniqueValue history. (b) An unserializable DuplicateValue history.

**Figure 2: Capturing SERIALIZABILITY via dependency graphs. Solid arrows denote known dependencies, whereas dashed arrows indicate uncertain ones.**

R3-2(3)

- $\text{RW} : \text{Key} \rightarrow 2^{\mathcal{T} \times \mathcal{T}}$  is such that  $\forall T, S \in \mathcal{T}. \forall x \in \text{Key}. T \xrightarrow{\text{RW}(x)} S \iff T \neq S \wedge \exists T' \in \mathcal{T}. T' \xrightarrow{\text{WR}(x)} T \wedge T' \xrightarrow{\text{WW}(x)} S$ .

We use  $\exists!$  to denote unique existence. For any component of a dependency graph  $\mathcal{G}$ , such as WW, we write it as  $\text{WW}_{\mathcal{G}}$ . When the key  $x$  in  $T \xrightarrow{R(x)} S$  is irrelevant or clear from context, we write  $T \xrightarrow{R} S$ , where  $R \in \{\text{WR}, \text{WW}, \text{RW}\}$ .

**Characterizing Serializability.** SERIALIZABILITY is characterized by the *existence* of an acyclic dependency graph, along with the internal consistency of each transaction, as axiomatized by INT [7]. The emphasis on existence is crucial: as external observers of a black-box database, verifiers cannot directly observe the internal execution order, e.g., WW between writes. Instead, they must infer a possible internal schedule. The existence of an acyclic dependency graph provides a witness that explains how the database could have scheduled the transactions in a way that satisfies SERIALIZABILITY.

In addition, the INT axiom ensures that a read on a key returns the same value as the most recent proceeding access—either a write or a read—to that key within the same transaction.

**THEOREM 2.4.** For a history  $\mathcal{H} = (\mathcal{T}, \text{SO})$ ,

$$\begin{aligned} \mathcal{H} \models \text{SER} &\iff \mathcal{H} \models \text{INT} \wedge \\ &\exists \text{WR}, \text{WW}, \text{RW}. \mathcal{G} = (\mathcal{H}, \text{WR}, \text{WW}, \text{RW}) \wedge \\ &((\text{SO}_{\mathcal{G}} \cup \text{WR}_{\mathcal{G}} \cup \text{WW}_{\mathcal{G}} \cup \text{RW}_{\mathcal{G}}) \text{ is acyclic}). \end{aligned}$$

**Example 2.5.** Figure 2 illustrates the dependency-graph-based characterization of SERIALIZABILITY using two example histories. For simplicity, we omit  $T_\perp$  in both cases. Figure 2a shows (the existence of) a serializable dependency graph with edges like  $T_2 \xrightarrow{\text{WW}(x)} T_1$ , which are constructed from a UniqueValue history.

In contrast, it is impossible to build an acyclic dependency graph from the DuplicateValue history in Figure 2b. Specifically, for key  $y$ , we obtain the dependencies  $T_2 \xrightarrow{\text{WR}(y)} T_1$  and  $T_3 \xrightarrow{\text{RW}(y)} T_2$  (due to  $T_\perp \xrightarrow{\text{WR}(y)} T_3$  and  $T_\perp \xrightarrow{\text{WW}(y)} T_2$ ). Now consider the transaction from which  $R(x, 1)$  in  $T_3$  reads its value. This leads to two possible WR edges (shown as dashed arrows):  $T_1 \xrightarrow{\text{WR}(x)} T_3$  and  $T_2 \xrightarrow{\text{WR}(x)} T_3$ . However, in both cases, the resulting dependency graph contains a cycle.

**Polygraphs.** A dependency graph extending a history represents *one* possible interpretation of the dependencies among the involved transactions, which may or may not satisfy SERIALIZABILITY. To capture *all* possible dependency scenarios—allowing a verifier to

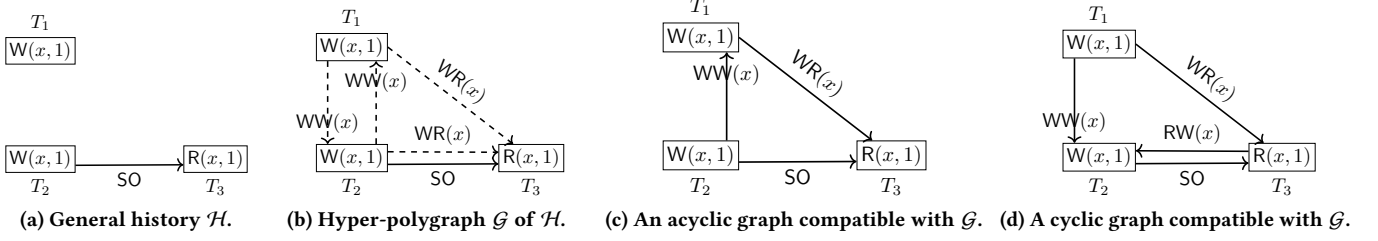


Figure 3: A general history  $\mathcal{H}$ , along with its hyper-polygraph  $\mathcal{G}$  and two compatible graphs.

R3-2(3)

check whether any of them satisfies SERIALIZABILITY—state-of-the-art tools [14, 32, 37] utilize *polygraphs* [28, 35]. Intuitively, a polygraph can be seen as a compact representation of a family of dependency graphs, each corresponding to a different possible internal execution consistent with the observed history.

**Definition 2.6.** A polygraph  $\mathcal{G} = (\mathcal{V}, \mathcal{E}, C)$  associated with a history  $\mathcal{H} = (\mathcal{T}, \text{SO})$  is a directed labeled graph  $(\mathcal{V}, \mathcal{E})$  called the *known graph*, together with a set  $C$  of *constraints*, such that

- $\mathcal{V}$  corresponds to all the transactions in  $\mathcal{H}$ ;
- $\mathcal{E} = \{(T, S, \text{SO}) \mid T \xrightarrow{\text{SO}} S\} \cup \{(T, S, \text{WR}) \mid T \xrightarrow{\text{WR}} S\}$ , where SO and WR, when used as the third component in a tuple, serve as edge labels (i.e., types of dependencies); and
- $C = \{(T_k, T_i, \text{WW}), (T_j, T_k, \text{RW}) \mid (T_i \xrightarrow{\text{WR}(x)} T_j) \wedge T_k \in \text{WriteTx}_x \wedge T_k \neq T_i \wedge T_k \neq T_j\}$ .

**Example 2.7.** According to the above definition, the polygraph associated with the history in Figure 2a consists of a known graph with nodes  $\{T_1, T_2, T_3\}$  and edges  $\{T_1 \xrightarrow{\text{WR}} T_3, T_2 \xrightarrow{\text{WR}} T_3\}$ , along with a set of two constraints  $\{(T_2, T_1, \text{WW}), (T_3, T_2, \text{RW})\}$  capturing the uncertainty in version order between  $T_1$  and  $T_2$ .

These constraints are then encoded as a SAT formula and solved using an SMT solver that supports theories like graph acyclicity, as we will see in Section 4. The acyclic dependency graph that satisfy SERIALIZABILITY in Figure 2a can be viewed as a solution produced by this solving process.

### 3 Hyper-Polygraphs

A dependency graph represents one possible execution of database transactions. A polygraph captures a family of such graphs by fixing the WR relation—an assumption that holds under Unique-Value, where each written value is unique and thus unambiguously matched by reads—while allowing uncertainty in the WW relation. However, this assumption, relied upon by all existing verifiers, does not always hold in practice, particularly under general workloads where the same value may be written multiple times, making read-from mappings ambiguous.

To address this representation problem, we introduce *hyper-polygraphs* as a generalization of polygraphs that additionally account for uncertainty in WR. Intuitively, a hyper-polygraph is a family of polygraphs, each corresponding to a distinct resolution of the WR relations, i.e., one polygraph per read-from mapping.

**Definition 3.1.** A hyper-polygraph  $\mathcal{G} = (\mathcal{V}, \mathcal{E}, C)$  for a history  $\mathcal{H} = (\mathcal{T}, \text{SO})$  is a directed labeled graph  $(\mathcal{V}, \mathcal{E})$ , referred

to as the known graph, together with a pair of constraint sets  $C = (C^{\text{WW}}, C^{\text{WR}})$ , where

- $\mathcal{V}$  is the set of nodes, corresponding to the transactions in  $\mathcal{H}$ ;
- $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V} \times \text{Type} \times \text{Key}$  is a set of edges, where each edge is labeled with a dependency type from  $\text{Type} = \{\text{SO}, \text{WR}, \text{WW}, \text{RW}\}$  and a key from  $\text{Key}$ ;<sup>2</sup>
- $C^{\text{WW}}$  is a constraint set over uncertain version orders, defined as  $C^{\text{WW}} = \left\{ \left\{ T \xrightarrow{\text{WW}(x)} S, S \xrightarrow{\text{WW}(x)} T \right\} \mid T \in \text{WriteTx}_x \wedge S \in \text{WriteTx}_x \wedge T \neq S \right\}$ ; and
- $C^{\text{WR}}$  is a constraint set over uncertain read-from mappings, defined as  $C^{\text{WR}} = \left\{ \bigcup_{T_i \vdash \text{W}(x,v)} \{T_i \xrightarrow{\text{WR}(x)} S\} \mid S \vdash \text{R}(x,v) \right\}$ .

Given two transactions  $T$  and  $S$ , a type  $T \in \text{Type}$ , and a key  $x \in \text{Key}$ , we also write the edge  $(T, S, T, x)$  as  $T \xrightarrow{T(x)} S$ .

**Example 3.2.** Consider the general history  $\mathcal{H}$  shown in Figure 3a. It consists of three transactions  $T_1, T_2$ , and  $T_3$ , with a session order edge  $T_2 \xrightarrow{\text{SO}} T_3$ . Both  $T_1$  and  $T_2$  write the same value 1 to the same key  $x$ . The hyper-polygraph  $\mathcal{G}$  constructed from  $\mathcal{H}$  includes the following two constraint sets:

- $C^{\text{WW}} = \left\{ \{T_1 \xrightarrow{\text{WW}(x)} T_2, T_2 \xrightarrow{\text{WW}(x)} T_1\} \right\}$ , representing the uncertainty in version order between  $T_1$  and  $T_2$  (shown as dashed edges in Figure 3b);
- $C^{\text{WR}} = \left\{ \{T_1 \xrightarrow{\text{WR}(x)} T_3, T_2 \xrightarrow{\text{WR}(x)} T_3\} \right\}$ , representing the uncertainty in read-from mappings for  $T_3$  (also shown as dashed edges in Figure 3b).

R3-2(3)

**Characterizing Serializability using Hyper-Polygraphs.** A hyper-polygraph for a DuplicateValue history can be viewed as a family of dependency graphs that are *compatible* with it, where each graph resolves the constraints by selecting exactly one WW or WR edge from the  $C^{\text{WW}}$  and  $C^{\text{WR}}$  constraint sets, respectively.

**Definition 3.3.** A directed labeled graph  $\mathcal{G}' = (\mathcal{V}', \mathcal{E}')$  is compatible with a hyper-polygraph  $\mathcal{G} = (\mathcal{V}, \mathcal{E}, C)$  if

- $\mathcal{V}' = \mathcal{V}$ ;
- $\mathcal{E}' \supseteq \mathcal{E}$  such that  $\forall x \in \text{Key}. \forall T, T', S \in \mathcal{V}'. (T', T, \text{WR}, x) \in \mathcal{E}' \wedge (T', S, \text{WW}, x) \in \mathcal{E}' \implies (T, S, \text{RW}, x) \in \mathcal{E}'$ ;
- $\forall C \in C^{\text{WW}}. |\mathcal{E}' \cap C| = 1$ ; and
- $\forall C \in C^{\text{WR}}. |\mathcal{E}' \cap C| = 1$ .

<sup>2</sup>For edges of type SO, the key component is irrelevant.



*Example 3.4.* Figures 3c and 3d depict two graphs compatible with the hyper-polygraph  $\mathcal{G}$  of the history  $\mathcal{H}$ . In Figure 3c,  $T_3$  reads  $x$  from  $T_1$ , which overwrites the value written by  $T_2$ . In contrast, in Figure 3d,  $T_3$  reads  $x$  from  $T_1$ , but the value is overwritten by  $T_2$ , resulting in  $T_3 \xrightarrow{RW(x)} T_2$ .

Based on Theorem 2.4, we obtain the following hyper-polygraph-based characterization of SERIALIZABILITY; the proof is provided in the Appendix [6].

**THEOREM 3.5.** *A history  $\mathcal{H}$  satisfies SERIALIZABILITY if and only if  $\mathcal{H} \models \text{INT}$  and there exists an acyclic graph compatible with the hyper-polygraph of  $\mathcal{H}$ .*

*Example 3.6.* The graph compatible with  $\mathcal{G}$  in Figure 3d contains a cycle:  $T_3 \xrightarrow{RW(x)} T_2 \xrightarrow{SO} T_3$ . In contrast, the compatible graph in Figure 3c is acyclic. According to Theorem 3.5, we conclude that  $\mathcal{H}_3$  satisfies SERIALIZABILITY.

## 4 Off-the-Shelf SMT Solving: A Baseline

To begin, we present a strong baseline approach for verifying SERIALIZABILITY using MonoSAT [4], an off-the-shelf SMT solver optimized for checking graph properties such as acyclicity. MonoSAT serves as the core engine for all state-of-the-art SMT-based isolation verifiers, including Cobra, PolySI, and Viper.

Given a history  $\mathcal{H}$  and following the workflow in Figure 1, we first construct its hyper-polygraph. This involves extracting the transaction set  $\mathcal{T}$ , the session order SO, and unique WR dependencies for the known graph, along with possible WW and WR dependencies as constraints. We then focus on the two key steps: *encoding* the hyper-polygraph into SAT formulas (Section 4.1) and *solving* them with MonoSAT (Section 4.2). We illustrate each step using the example history  $\mathcal{H}$  shown in Figure 3a.

*Preliminaries.* In propositional logic, a boolean variable  $v$  can take the value true or false. A *literal*  $l$  refers to a variable  $v$  or its negation  $\neg v$ . A *clause*  $C$  is a disjunction of one or more literals, e.g.,  $C = l_1 \vee l_2 \vee \dots \vee l_n$ . A *formula*  $\mathcal{F}$  is constructed using literals and the logical connectives  $\wedge, \vee, \neg$ , or  $\implies$ . Typically, we represent formulas in conjunctive normal form (CNF), where  $\mathcal{F}$  is a conjunction of multiple clauses:  $\mathcal{F} = C_1 \wedge C_2 \wedge \dots \wedge C_m$ . An *assignment* of a variable  $v$  is either true or false, represented as  $v$  or  $\neg v$ , respectively. A *model*  $\mathcal{M}$  for a formula  $\mathcal{F}$  is a set of assignments to the boolean variables appearing in  $\mathcal{F}$  that makes  $\mathcal{F}$  true. If  $\mathcal{M}$  does not specify assignments for all variables in  $\mathcal{F}$ , it is referred to as a *partial model*.

### 4.1 Encoding

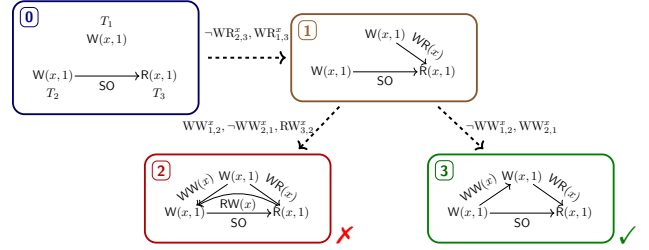
We represent the existence of each edge in the hyper-polygraph  $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{C})$  using a boolean variable. First, the known graph of  $\mathcal{G}$  is encoded by the formula

$$\Phi_{KG} = \text{SO}_{2,3}$$

where the variable  $\text{SO}_{2,3}$ , set to true, indicates the presence of the SO edge  $T_2 \xrightarrow{SO} T_3$ .

Second, the  $\mathcal{C}^{\text{WW}}$  constraints of  $\mathcal{G}$  are encoded by the formula

$$\Phi_{C^{\text{WW}}} = (\text{WW}_{1,2}^x \vee \text{WW}_{2,1}^x) \wedge (\neg \text{WW}_{1,2}^x \vee \neg \text{WW}_{2,1}^x)$$



**Figure 4: An example solving process for  $\mathcal{H}$  in Figure 3.**

where the variables  $\text{WW}_{1,2}^x$  and  $\text{WW}_{2,1}^x$  represent the existence of the edges  $T_1 \xrightarrow{WW(x)} T_2$  and  $T_2 \xrightarrow{WW(x)} T_1$ , respectively. This formula enforces that exactly one of the two variables is assigned true, ensuring a total order between the two conflicting writes.

Moreover, the  $\mathcal{C}^{\text{WR}}$  constraints of  $\mathcal{G}$  are encoded by the formula

$$\Phi_{C^{\text{WR}}} = (\text{WR}_{1,3}^x \vee \text{WR}_{2,3}^x) \wedge (\neg \text{WR}_{1,3}^x \vee \neg \text{WR}_{2,3}^x)$$

where the variables  $\text{WR}_{1,3}^x$  and  $\text{WR}_{2,3}^x$  denote the presence of the edges  $T_1 \xrightarrow{WR(x)} T_3$  and  $T_2 \xrightarrow{WR(x)} T_3$ , respectively. This formula enforces that exactly one of these edges holds, ensuring a unique read-from relation for  $T_3$ .

Finally, the encoding of the derivation rule for RW edges in  $\mathcal{G}$  is captured by the formula

$$\begin{aligned} \Phi_{\text{RW}} &= (\text{WW}_{1,2}^x \wedge \text{WR}_{1,3}^x \implies \text{RW}_{3,2}^x) \wedge (\text{WW}_{2,1}^x \wedge \text{WR}_{2,3}^x \implies \text{RW}_{3,1}^x) \\ &\iff (\neg \text{WW}_{1,2}^x \vee \neg \text{WR}_{1,3}^x \vee \text{RW}_{3,2}^x) \wedge (\neg \text{WW}_{2,1}^x \vee \neg \text{WR}_{2,3}^x \vee \text{RW}_{3,1}^x) \end{aligned}$$

where  $\text{RW}_{3,1}^x$  and  $\text{RW}_{3,2}^x$  represent the existence of the RW edges  $T_3 \xrightarrow{RW(x)} T_1$  and  $T_3 \xrightarrow{RW(x)} T_2$ , respectively. This encoding ensures that an RW edge is derived only when both the corresponding WW and WR edges are present.

Overall, the complete encoding for  $\mathcal{G}$  is given by

$$\Phi_{\mathcal{G}} = \Phi_{KG} \wedge \Phi_{C^{\text{WW}}} \wedge \Phi_{C^{\text{WR}}} \wedge \Phi_{\text{RW}}.$$

### 4.2 Solving

Each satisfying assignment of  $\Phi_{\mathcal{G}}$  corresponds to a graph  $\mathcal{G}'$  that is compatible with  $\mathcal{G}$ . To ensure that  $\mathcal{G}'$  is acyclic, we assert the predicate  $\text{Acyclic}(\mathcal{G}')$  in MonoSAT. Hence, the history  $\mathcal{H}$  is serializable if and only if the formula  $\Phi_{\mathcal{G}} \wedge \text{Acyclic}(\mathcal{G}')$  is satisfiable.

MonoSAT implements the *Conflict-Driven Clause Learning with Theory* (CDCL(T)) framework [27] to decide the satisfiability of SAT formulas augmented with theory predicates. It combines a SAT solver, which searches for a satisfying assignment to the boolean formula (e.g.,  $\Phi_{\mathcal{G}}$ ), with a theory solver, which ensures that the asserted predicates (e.g.,  $\text{Acyclic}(\mathcal{G}')$ ) hold under the current assignment. As the SAT solver assigns variables, the theory solver incrementally updates a graph  $\mathcal{G}^*$  by adding edges for variables set to true. If a predicate is violated, a conflict clause is generated and learned by the SAT solver to prevent revisiting the same conflict.

The solving process may vary depending on the order in which variables are assigned. Figure 4 illustrates one such procedure for  $\mathcal{G}$ , with the graph  $\mathcal{G}^*$  progressing through four stages.<sup>3</sup> Initially, only  $\text{SO}_{2,3}$  is assigned true, yielding [0]. Suppose the SAT solver next assigns  $\neg \text{WR}_{1,3}^x$ . Then  $\text{WR}_{2,3}^x$  must be true to satisfy the clause

<sup>3</sup>We also illustrate in the Appendix [6] how the SAT solver and the theory solver collaborate in CDCL(T) for this example.

$WR_{1,3}^x \vee WR_{2,3}^x$ . This is also known as *unit propagation* [8] that occurs when a clause becomes unit, i.e., only one literal remains unassigned ( $WR_{1,3}^x$  in this case), forcing the literal to be assigned true to satisfy the clause. The theory solver adds the edge  $T_1 \xrightarrow{WR(x)} T_3$  to  $G^*$ , leading to ①. No cycles are detected at this point.

Next, assigning  $WW_{1,2}^x$  triggers  $\neg WW_{2,1}^x$  due to mutual exclusion, and then  $RW_{3,2}^x$  via the clause  $\neg WW_{1,2}^x \vee \neg WR_{1,3}^x \vee RW_{3,2}^x$ , which leads to the updated graph  $G^*$  in ②. At this point, the theory solver detects a cycle:  $T_2 \xrightarrow{SO} T_3 \xrightarrow{RW(x)} T_2$  (see also Figure 3d). It then returns a conflict clause  $\neg RW_{3,2}^x \vee \neg SO_{2,3}$  to the SAT solver, indicating that removing either edge would break the cycle. The SAT solver then backtracks from  $WW_{1,2}^x$ , assigns its negation  $\neg WW_{1,2}^x$ , and propagates  $WW_{2,1}^x$ . This results in ③, where all variables are assigned and no cycles are present. Hence, an acyclic compatible graph is found (see also Figure 3c), and the history  $\mathcal{H}$  is serializable.

## 5 Workload Characteristics for SMT Solving

General-purpose SMT solvers like MonoSAT are agnostic to the characteristics of database workloads, thereby missing opportunities for targeted optimization. In contrast, we hypothesize that leveraging workload-specific characteristics can significantly improve solving efficiency for isolation verification. This section presents two key observations that motivate our tailored SMT strategies.

### 5.1 Prevalence of 2-width Cycles

A key bottleneck arises in the solving stage from “bad” guesses that introduce cycles in the dependency graph. When this occurs, the solver must backtrack and explore alternative choices, which can be costly. To mitigate this, prior work [14, 32, 37] introduces *pruning*, a preprocessing technique that identifies and precludes infeasible WW constraints by analyzing reachability *before* the actual solving process starts. For example, consider a WW constraint  $\{T_1 \xrightarrow{WW(x)} T_2, T_2 \xrightarrow{WW(x)} T_1\}$ . If  $T_2$  is already reachable from  $T_1$ , then choosing  $T_2 \xrightarrow{WW(x)} T_1$  would result in a cycle. Hence, this choice can be safely pruned, eliminating the need for the solver to explore it further. In the context of SAT solving, pruning corresponds to adding a conflict clause with a single WW variable, e.g.,  $\neg WW_{2,1}^x$  in this case, to the encoding for  $\Phi_{\mathcal{G}}$  (Section 4.1).

Essentially, pruning shifts the cost of resolving search-time conflicts to a pre-solving analysis phase. This raises a natural question: can more of the solving overhead be offloaded to preprocessing? In other words, is there a sweet spot that balances the cost of pre-computation with the benefit of reduced solving effort, thereby improving overall performance?

To explore this, we adapt the notion of *cycle width* [10, 13], which we define as the number of *unique* WW and WR variables appearing in a conflict clause. While conventional pruning relies on unit clauses (i.e., 1-width cycles), broader pruning can be achieved by encoding larger cycles using multi-variable clauses. To avoid redundancy, we count each variable only once and disregard SO variables, which are always true, and RW variables, which can be derived from WW and WR.

We empirically measure the widths of conflict cycles encountered during solving across six representative benchmarks, including TPC-C and YCSB-like transaction workloads with varying read proportions (see Section 7.1 for details). Each benchmark consists of 1000 transactions. For each conflict, we record the *minimal* cycle width, as it may correspond to multiple cycles depending on the search path. As shown in Figure 5, 2-width cycles dominate across all benchmarks. This suggests that many conflicts could be prevented by pre-encoding 2-variable clauses. We leverage this insight in Section 6.1, incorporating 2-width cycle encodings into the SAT formula  $\Phi_{\mathcal{G}}$  to strike a balance between pre-solving effort and solving efficiency (further validated in Section 7.3.3).

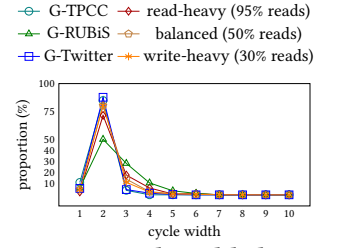


Figure 5: Cycle width distribution across conflicts.

### 5.2 Polarity as Schedule Reconstruction

Another major source of inefficiency in SMT solving stems from poor polarity choices when assigning decision variables. A key insight we introduce is that each polarity decision—whether to include or exclude a dependency edge—implicitly defines a transaction schedule. Suboptimal choices can conflict with the actual, though hidden (due to the black-box setting), schedule enforced by the database, leading to dependency cycles and costly backtracking.

General-purpose solvers such as MonoSAT treat all variables uniformly and assign polarity without regard to the execution context. In contrast, we observe that many dependencies, including session order SO and uniquely determined read-from relation WR, are already known from the observable history and can be leveraged to guide more informed polarity decisions. Ideally, if the internal transaction schedule were accessible, one could align polarity assignments perfectly to avoid conflicts. While this is infeasible in black-box settings, the visible dependencies still capture a significant portion of the execution order. We hypothesize that exploiting this partial order can substantially reduce conflict rates and backtracking overhead. The following example illustrates this intuition.

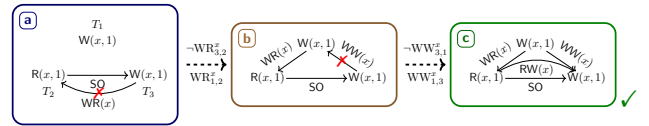


Figure 6: Guiding polarity picking via known dependencies.

**Example 5.1.** In Figure 6, the solver must resolve two constraints:  $\{T_1 \xrightarrow{WR(x)} T_2, T_3 \xrightarrow{WR(x)} T_2\}$  and  $\{T_1 \xrightarrow{WW(x)} T_3, T_3 \xrightarrow{WW(x)} T_1\}$ . Initially, in ①, the solver must decide between two possible read-from candidates for  $T_2$ . By recognizing the known session order  $T_2 \xrightarrow{SO} T_3$ , the solver correctly assigns  $WR_{3,2}^x = \text{false}$ , avoiding a potential cycle between  $T_2$  and  $T_3$ . The solver then proceeds to ②, where it must resolve the WW constraint. Again, guided by the inferred dependency  $T_1 \xrightarrow{WR(x)} T_3$ , it avoids introducing a cycle, e.g.,  $T_1 \xrightarrow{WR(x)} T_2 \xrightarrow{SO} T_3 \xrightarrow{WW(x)} T_1$ , by choosing  $WW_{3,1}^x = \text{false}$ .

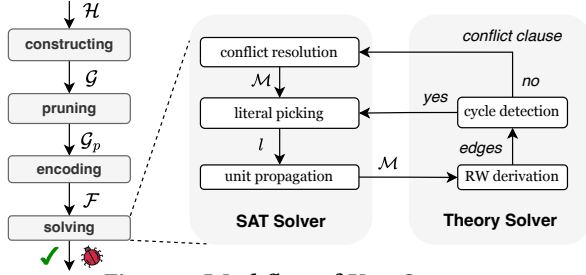


Figure 7: Workflow of VERISTRONG.

This example underscores the importance of polarity decisions: in large histories, a single incorrect polarity choice may introduce an edge that contradicts the actual schedule. Consequently, the solver is misled into exploring an exponentially larger search space, incurring substantial overhead due to cascading conflicts and backtracking. Motivated by this insight, the next section introduces a polarity picking strategy—integrated into our dedicated SMT solver—that exploits the observable partial order in the execution history. We evaluate its impact in Section 7.3.4.

## 6 VERISTRONG with Tailored SMT Solving

VERISTRONG builds upon the baseline algorithm described in Section 4. Motivated by our observations in Section 5, it incorporates two key optimizations: efficient handling of 2-width cycles (Section 6.1) and polarity picking for decision variables (Section 6.2).

Figure 7 shows the high-level workflow of VERISTRONG, comprising four main stages: (i) *constructing* the hyper-polygraph  $\mathcal{G}$  from the input history  $\mathcal{H}$ , (ii) *pruning* infeasible constraints in  $\mathcal{G}$ , (iii) *encoding* the pruned graph  $\mathcal{G}_p$  into a SAT formula  $\mathcal{F}$ , and (iv) *solving*  $\mathcal{F}$ . The corresponding pseudo-code is given in Algorithm 1, which we refer to alongside the figure to explain the procedure.

The remainder of Algorithm 1 details the solving phase, specifically the interaction between the SAT frontend (SATSOLVE, Line 11) and the theory backend (THEORYSOLVE, Line 26). Let  $G^*$  denote the current known graph maintained in the theory solver. Initially,  $G^*$  is set to the hyper-polygraph  $\mathcal{G}$  constructed from  $\mathcal{H}$ . In each iteration, the SAT solver selects a decision variable  $v$  (Line 14) and its polarity, yielding a decision literal  $l$  (Line 15). This literal is then unit propagated, producing a partial assignment  $\mathcal{M}$  (Line 16) that is passed to the theory solver for conflict checking (Line 17).

Specifically, the theory solver first derives RW edges from the current WW and WR edges (Line 27) and checks whether the updated graph  $G^*$  contains any cycles (Line 28). If a cycle is found, it generates a conflict clause and returns it to the SAT solver (Line 30). The SAT solver then resolves the conflict, either terminating with UNSAT (Line 20) or backtracking and updating  $\mathcal{M}$  (Line 22). If no cycle is detected, the solver continues by selecting the next decision variable. Once all variables have been assigned without encountering conflicts (Line 13), the solver reports SAT (or serializable).

### 6.1 Small-Width Cycle Preprocessing

This optimization aims to shift the burden of handling conflicts within the solver to a lightweight pre-solving analysis that focuses on small-width cycles. Intuitively, this preprocessing step acts as a “sanity check” before solving: it rules out choices that are guaranteed

#### Algorithm 1 The VERISTRONG algorithm

$\mathcal{H}$ : the input history  
 $G^* = (\mathcal{V}, E^*)$ : the current graph maintained in the theory solver; initialized as the hyper-polygraph  $\mathcal{G}$  constructed from  $\mathcal{H}$   
 $\text{Vars}$ : a set of boolean variables managed by the SAT solver; initially  $\emptyset$   
 $\text{Clauses}$ : a set of clauses managed by the SAT solver; initially  $\emptyset$   
Both  $\text{Vars}$  and  $\text{Clauses}$  are constructed in ENCODE (Line 5).

```

1: function VERIFYSER( $\mathcal{H}$ )
2:    $\mathcal{G} \leftarrow \text{CONSTRUCT}(\mathcal{H})$  ▷ Alg. 4 in the Appendix [6]
3:   if  $\neg \text{PRUNE}(\mathcal{G})$  ▷ Alg. 5 in the Appendix; see also Section 6.1.1
4:   return false
5:    $\mathcal{F} \leftarrow \text{ENCODE}(\mathcal{G}_p)$  ▷ Alg. 7 in the Appendix; see also Section 6.1.2
6:   return SOLVE( $\mathcal{F}, \mathcal{G}_p$ )

7: function SOLVE( $\mathcal{F}, \mathcal{G}$ )
8:   for  $T \xrightarrow{T(x)} S \in \mathcal{E}$ 
9:    $E^* \leftarrow E^* \cup \{T \rightarrow S\}$ 
10:  return SATSOLVE( $\mathcal{F}, \mathcal{G}$ )

11: function SATSOLVE( $\mathcal{F}, \mathcal{G}$ )
12:   $\mathcal{M}_{\text{total}} \leftarrow \emptyset$ 
13:  while  $|\mathcal{M}_{\text{total}}| \neq |\text{Vars}|$ 
14:     $v \leftarrow \text{CHOOSEDECISIONVAR}(\mathcal{M}_{\text{total}}, \text{Vars})$ 
15:     $l \leftarrow \text{PICKPOLARITY}(v)$  ▷ Alg. 3 in Section 6.2
16:     $\mathcal{M} \leftarrow \text{UNITPROPAGATE}(l, \mathcal{F})$ 
17:     $(\text{ret}, \text{conflict\_clause}) \leftarrow \text{THEORYSOLVE}(\mathcal{M})$ 
18:    if  $\neg \text{ret}$  ▷ a conflict is detected
19:      if  $\neg \text{RESOLVECONFLICT}(\mathcal{M}_{\text{total}}, \mathcal{M}, \text{conflict\_clause})$ 
20:        return UNSAT
21:      else
22:         $\mathcal{M}_{\text{total}} \leftarrow \text{BACKTRACK}(\mathcal{M}_{\text{total}}, \mathcal{M}, \text{conflict\_clause})$ 
23:        continue, goto Line 13
24:     $\mathcal{M}_{\text{total}} \leftarrow \mathcal{M}_{\text{total}} \cup \mathcal{M}$ 
25:  return SAT

26: function THEORYSOLVE( $\mathcal{M}$ )
27:   $\text{edges} \leftarrow \text{DERIVERWEDGES}(\mathcal{M})$ 
28:  if  $(E^* \cup \text{edges})$  is cyclic ▷ use the PK algorithm [29]
29:     $\text{conflict\_clause} \leftarrow \text{GENCONFLICTCLAUSE}(E^*, \text{edges})$ 
30:    return (false, conflict\_clause)
31:  else
32:     $E^* \leftarrow E^* \cup \text{edges}$ 
33:    return (true, null)

```

to fail (i.e., those that create cycles as exemplified in Section 5.1), preventing the solver from wasting effort on them later. It consists of two components: (i) an aggressive pruning phase that eliminates all 1-width cycles through reachability analysis, and (ii) a proactive encoding of 2-width cycles into the SAT formula, enabling early conflict detection via unit propagation during solving.

**6.1.1 Eliminating 1-Width Cycles.** We extend the pruning techniques from prior work [14, 32, 37], which focus on WW constraints, by also pruning WR constraints and their derived RW counterparts. The following examples illustrate the core ideas of our approach.

*Case I: Pruning WW Constraints.* Consider the two transactions  $T$  and  $S$  in Figure 8a, both writing to the same key. If  $T$  is already reachable from  $S$  in the current known graph, then adding  $T \xrightarrow{\text{WW}(x)}$

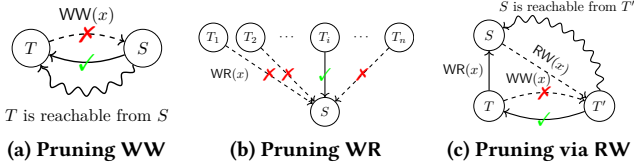


Figure 8: Cases for pruning 1-width cycles.

$S$  would introduce a 1-width cycle. This infeasible option can thus be pruned; the valid edge  $S \xrightarrow{WW(x)} T$  is added to the known graph.

**Case II: Pruning WR Constraints.** We can similarly prune a WR edge if its inclusion would introduce a 1-width cycle in the known graph. Consider the scenario in Figure 8b, where a transaction  $S$  reads from key  $x$  and has  $n$  candidate writers  $T_1, \dots, T_n$ . If  $n - 1$  of these candidates have been eliminated due to cycle formation, the remaining writer must be the only feasible source. In this case, we can safely add the edge  $T_i \xrightarrow{WR(x)} S$  to the known graph.

**Case III: Pruning via Derived RW Edges.** Derived edges may also lead to cycles. In Figure 8c, assume that adding  $T \xrightarrow{WW(x)} T'$  induces  $S \xrightarrow{RW(x)} T'$  (via  $T \xrightarrow{WR(x)} S$ ). If  $S$  is already reachable from  $T'$ , this creates a cycle, so  $T \xrightarrow{WW(x)} T'$  is pruned.

VERISTRONG iteratively applies this pruning process until the known graph stabilizes, i.e., all 1-width cycles have been eliminated. The complete pseudo-code is provided in the Appendix [6].

**6.1.2 Encoding 2-width Cycles.** To prevent conflicts caused by 2-width cycles during solving, VERISTRONG encodes them into SAT formulas. The corresponding pseudo-code is provided in Algorithm 2. We consider two scenarios, as also illustrated in Figure 9.

**Case I: Canonical 2-Width Cycles.** This case illustrates a canonical 2-width cycle formed by two known paths, as shown in Figure 9a. Let  $T \rightsquigarrow T'$  and  $S' \rightsquigarrow S$  be known dependencies in the graph, where  $\rightsquigarrow$  denotes reachability between transactions. Adding both candidate edges  $S \rightarrow T$  and  $T' \rightarrow S'$  would complete a cycle. We prevent this by adding the clause  $\neg v_{S,T} \vee \neg v_{T',S'}$  to the formula, where  $v_{S,T}$  and  $v_{T',S'}$  are the corresponding boolean variables.

**Case II: Derived RW Cycles.** In Figure 9b, assume  $T' \rightsquigarrow S$  is known. Adding both  $T \xrightarrow{WR(x)} S$  and  $T \xrightarrow{WW(x)} T'$  would derive  $S \xrightarrow{RW(x)} T'$ , forming a cycle. To avoid this, we encode the clause:  $\neg WW_{T,T'}^x \vee \neg WR_{T,S}^x$ .

While longer cycles could, in theory, also be encoded, enumerating clauses over  $k$  boolean variables incurs a time complexity of  $O(|Vars|^k)$ , making it impractical for large  $k$ . Our analysis shows that 2-width clauses capture the majority of conflicts effectively (Figure 15). Hence, we focus on encoding only 2-width cycles, striking a balance between preprocessing overhead and solving efficiency.

**6.1.3 Why our preprocessing works.** Our encoding of 2-width cycles does not introduce additional constraints, as the encoded cycles cannot appear in a valid model. Hence, equisatisfiability with the original encoding is preserved. Moreover, pruning can be viewed as a specialization of encoding 1-width cycles. This process is lifted from solving to the manipulation of hyper-polygraphs, and involves

## Algorithm 2 Encoding 2-width cycles

```

1: function ENCODINGCYCLESOFWIDTH2( $\mathcal{V}, \mathcal{E}$ )
2:    $\mathcal{R} \leftarrow \text{REACHABILITY}(\mathcal{V}, \mathcal{E})$ 
3:   for  $v_1, v_2 \neq v_1 \in \text{Vars}$ 
4:     if  $\text{CANONICALCYCLE}(v_1, v_2, \mathcal{R}) \vee \text{RWCYCLE}(v_1, v_2, \mathcal{R})$ 
5:        $\text{Clauses} \leftarrow \text{Clauses} \cup \{\neg v_1 \vee \neg v_2\}$ 
6:   function CANONICALCYCLE( $v_1, v_2, \mathcal{R}$ )
7:     let  $(S, T) \leftarrow v_1, (T', S') \leftarrow v_2$ 
8:     return  $(T, T') \in \mathcal{R} \wedge (S, S') \in \mathcal{R}$ 
9:   function RWCYCLE( $v_1, v_2, \mathcal{R}$ )
10:    return  $\exists x. \exists T, S, T'. T' \neq T \wedge (T', S) \in \mathcal{R} \wedge ((v_1 \triangleq WW_{T,T'}^x \wedge v_2 \triangleq WR_{T,S}^x) \vee (v_2 \triangleq WW_{T,T'}^x \wedge v_1 \triangleq WR_{T,S}^x))$ 

```

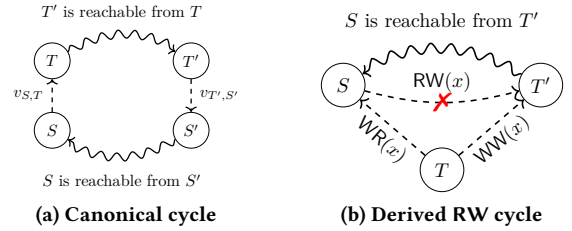


Figure 9: Cases for encoding 2-width cycles.

two key steps: (i) eliminating infeasible edges in  $\mathcal{C}$  (a specialization of conflict detection and learning), and (ii) adding the final edge to the known graph ( $\mathcal{V}, \mathcal{E}$ ) (a specialization of propagation). Correctness of pruning also follows from equisatisfiability with the original encoding. We defer the formal proofs to the Appendix [6].

## 6.2 Order-Guided Polarity Picking

VERISTRONG leverages the current partial order of the dependency graph to guide polarity picking, as motivated in Section 5.2. To achieve this, we incorporate the *pseudo-topological order* maintained by the PK algorithm [29], which is originally developed for dynamic cycle detection and also adopted by MonoSAT. The full polarity picking procedure is given in Algorithm 3.

## Algorithm 3 Pseudo-topological-order-guided polarity picking

```

1: function PICKPOLARITY( $v$ )
2:   let  $(\text{from}, \text{to}) \leftarrow v$ 
3:   if  $\text{level}(\text{from}) < \text{level}(\text{to})$ 
4:     return  $v$  ▷ positive polarity
5:   else
6:     return  $\neg v$  ▷ negative polarity

```

The PK algorithm dynamically maintains a level  $\text{level}(\cdot)$  for each vertex, which approximates its position in a topological ordering. When a candidate edge  $(\text{from}, \text{to})$  is considered, the algorithm first checks whether  $\text{level}(\text{from}) < \text{level}(\text{to})$ . If so, the edge is guaranteed not to introduce a cycle and is immediately accepted. If not, the algorithm performs a reachability check to ensure that no cycle would be formed, and if the edge is added, it updates the levels of affected vertices to maintain consistency.

This pseudo-topological order reflects the known partial order in the current graph: every edge that has been included satisfies  $\text{level}(\text{from}) < \text{level}(\text{to})$ . We exploit this order to guide polarity selection. For a decision variable  $v$  representing a candidate edge

R3-2(3)

R3-2(3)

Meta-1



(*from*, *to*), we assign it a *positive polarity* (i.e., include the edge) when  $\text{level}(\text{from}) < \text{level}(\text{to})$ , and a *negative polarity* (i.e., exclude the edge) otherwise.

This heuristic encourages decisions that respect the observed partial order and are thus less likely to introduce cycles. Although it cannot guarantee cycle-freeness in all cases, especially when the topological approximation is incomplete, it serves as a practical guide for reducing backtracking. If a polarity choice later results in a conflict, the solver will backtrack and try the opposite polarity. Since this heuristic does not alter the search space of CDCL(T), correctness is preserved. The proof can be found in the Appendix [6].

Meta-1

### 6.3 Complexity Analysis

**Constructing.** Let  $\|C\| = \sum_{\text{cons} \in C^{\text{WW}} \cup C^{\text{WR}}} |\text{cons}|$ . The construction

phase requires enumerating all edges in  $C$ , yielding a time complexity of  $O(\|C\|)$ . Let  $k$  be the number of keys and  $n$  the number of transactions. In the worst case,  $O(\|C\|) = O(kn^2)$ .

**Pruning.** Pruning proceeds in iterative passes. Each pass first computes reachability and then enumerates constraints in  $C$  to test the three pruning cases shown in Figure 8. Computing reachability costs  $O(|V| \cdot |\mathcal{E}|)$ , where  $(V, \mathcal{E})$  is the known graph. With reachability cached, testing each pruning case costs only  $O(1)$ . The main bottleneck is enumerating possible RW edges, which can be characterized by the number of tuples  $(T, T', S, x)$  with  $T, T', S \in \mathcal{T}, x \in \text{Key}, T, T' \vdash W(x, \_)$  and  $S \vdash R(x, \_)$ . This costs  $O(\sum_{x \in \text{Key}} |\text{WriteTx}_x|^2 \cdot |\text{ReadTx}_x|)$ , where  $\text{ReadTx}_x = \{T \mid T \vdash R(x, \_)\}$ . Let  $N_p$  be the number of pruning passes. The overall complexity of pruning is then  $O(N_p \cdot (|V| \cdot |\mathcal{E}| + \sum_{x \in \text{Key}} |\text{WriteTx}_x|^2 \cdot |\text{ReadTx}_x|))$ . Experiments

show that  $N_p$  is typically small (less than 10), and can thus be treated as  $O(1)$ . In the worst case, pruning has complexity  $O(kn^3)$ .

**Encoding.** Let  $C_p$  denote the constraints in the pruned hyperpolygraph  $\mathcal{G}_p$ . The encoded formula consists of two parts: the constraints in  $C_p$  and the clauses for 2-width cycles. The bottleneck lies in enumerating edge pairs to detect 2-width cycles (see Algorithm 2), which costs  $O(\|C_p\|^2)$ . Hence, the total encoding complexity is  $O(\|C_p\|^2)$ , or  $O(k^2n^4)$  in the worst case.

**Solving.** The final stage, SMT solving, is NP-hard in general. Although prior research has analyzed SMT solvers and established lower bounds in certain settings [30], the precise complexity of our setting remains open [19].

Despite this inherent theoretical hardness, experiments show that our approach achieves high efficiency on practical workloads, where worst-case scenarios rarely arise (Section 7). This reflects the motivating insight of our work: database workload characteristics can be leveraged to build efficient and practical verifiers.

## 7 Experiments

We have implemented our algorithm in a tool called VERISTRONG, built on top of MiniSat [9], a minimalistic and open-source SAT solver. We selected MiniSat due to its simplicity and extensibility, which allowed us to integrate our optimization strategies effectively.

To support the optimization of small-width cycles (Section 6.1), we compute reachability in the known graph using dynamic programming. Specifically, VERISTRONG maintains a 0-1 matrix of size  $|\mathcal{T}| \times |\mathcal{T}|$ , where each entry records whether one transaction is reachable from another. By traversing nodes in reverse topological order, it incrementally updates each node’s reachable set: for an edge (*from*, *to*), it unions the reachability set of *to* into that of *from*. We implement the matrix using bit vectors and bitwise operations, providing a lightweight alternative to GPU-based solutions [32]. Overall, VERISTRONG has approximately 5k lines of C++ code.

We conduct an extensive evaluation of VERISTRONG, focusing on its checking efficiency, while also assessing its bug-finding effectiveness. Specifically, we aim to answer the following questions:

- **Efficiency:** How efficiently does VERISTRONG perform across various general workloads (Section 7.2.1)? Can it outperform existing tools (Section 7.2.2)? Does it scale to large histories (Section 7.2.3)? How do its individual components contribute to the overall performance (Section 7.3)?
- **Effectiveness:** How effective is VERISTRONG at uncovering isolation bugs in production database systems, particularly those related to DuplicateValue (Section 7.4)?

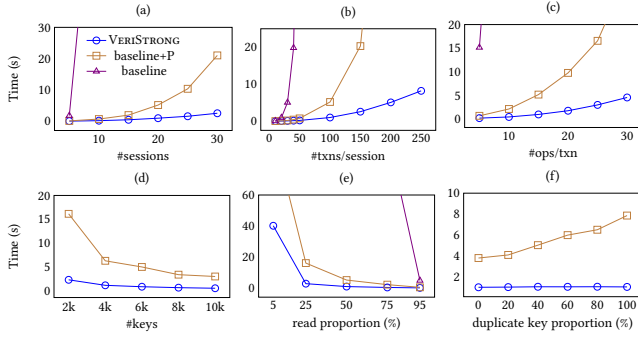
### 7.1 Benchmarks and Experimental Setup

We evaluate VERISTRONG and competing verifiers using two categories of benchmarks. The first category comprises YCSB-like transactional workloads, produced by a parametric workload generator built on top of PolySI [14]. This generator supports several configurable parameters, with default values shown in parentheses: the number of client sessions (20), the number of transactions per session (100), the number of read/write operations per transaction (20), the overall read proportion (50%), the total number of keys (5k), and the proportion of keys with duplicate write values (50%).

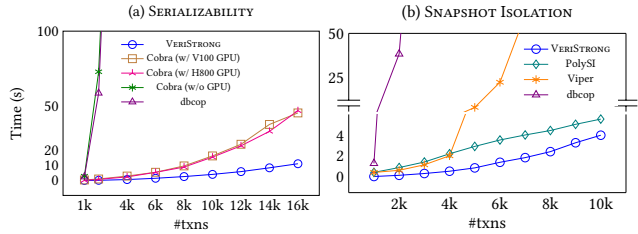
We characterize duplicate write values using a Zipfian distribution, where the parameter  $\theta$  (0.5) controls the degree of duplication, and  $N$  (100) defines the size of value space. We then generate a set of representative workloads, including RH (read-heavy, 95% reads), WH (write-heavy, 70% writes), and BL (balanced, 50% reads). These workloads align with UniqueValue ones commonly used in prior work. In addition, we include HD, a variant of BL with a high degree of duplication ( $\theta = 1.5$ ). Each workload is executed on a PostgreSQL v15 instance to produce transactional histories, each containing at least 10k transactions and 80k operations.

The second category comprises application-level benchmarks: TPC-C [33], a standard OLTP benchmark configured with 1 warehouse, 10 districts, 30k customers, and 5 transaction types; RUBiS [31], an auction-based system with 20k users and 200k items; and TwitterClone [17], a blogging application with Zipfian key access patterns. We use a PostgreSQL instance to generate general transactional histories from these applications, which we refer to as G-TPCC, G-RUBiS, and G-Twitter, respectively. Notably, both RUBiS and TwitterClone naturally generate duplicate write values. However, prior work [14, 32, 37] disabled this behavior to enforce the UniqueValue assumption. In contrast, we preserve these duplicate-value semantics to more accurately reflect real-world behavior.

Unless otherwise stated, all experiments are performed on a local machine equipped with an Intel 13th Gen i5 CPU and 32GB RAM.



**Figure 10: Verification time comparison across a range of general workloads. Timeout (60s) data points are not plotted.**



**Figure 11: Comparison with existing verifiers under Unique-Value histories using the BlindW-RW datasets (with 10k keys in (a) and 2k keys in (b)).**

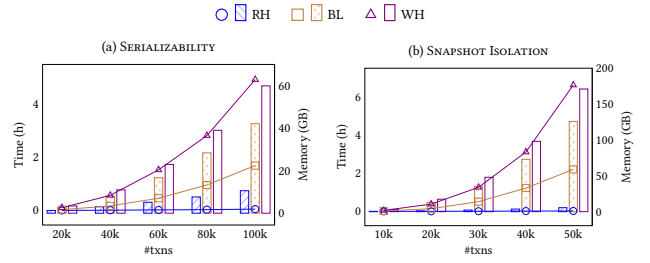
## 7.2 Performance Evaluation

**7.2.1 Comparison with Baselines.** Our first set of experiments compares VERiSTRONG with the baseline algorithm (Section 4) across various general workloads. We also include a stronger variant, baseline+P, which incorporates the pruning technique for WW constraints (Section 5.1).

The experimental results are shown in Figure 10, omitting data points that exceed the 60s timeout. VERiSTRONG consistently outperforms both the baseline and its stronger variant. Specifically, under increased concurrency, such as more sessions (a), more transactions per session (b), more operations per transaction (c), and smaller key space (d), the two baselines suffer from exponential verification time, while VERiSTRONG incurs only moderate overhead. Furthermore, in scenarios with increased uncertainty in WW and WR dependencies, such as write-heavy workloads (e) and higher proportions of duplicate keys (f), VERiSTRONG remains the fastest while maintaining fairly stable verification efficiency.

In addition, we measure the memory usage of all three tools under the same settings as in Figure 10. The trends are similar: VERiSTRONG uses less memory than both baselines across a range of workloads. Memory plots are provided in the Appendix [6].

**7.2.2 Comparison with State-of-the-Art.** Our second set of experiments evaluate VERiSTRONG against four state-of-the-art verifiers: Cobra [32] for SERIALIZABILITY, PolySI [14] and Viper [37] for SNAPSHOT ISOLATION, and dbcop [5] for both. Since Cobra supports GPU acceleration for pruning, we also include its GPU-enabled versions in our evaluation. Note that dbcop is included as a representative of non-SMT-based verifiers, which rely on graph traversal algorithms, such as depth-first search, to detect cycles.



**Figure 12: VERiSTRONG's performance under large workloads. Time and memory usage are plotted in lines and bars.**

For a fair comparison, we use the BlindW-RW dataset adopted in prior work [32, 37]. This dataset exclusively consists of UniqueValue histories, as all the above verifiers are restricted to such input. Each history contains an even mix of read-only and write-only transactions, with each transaction comprising eight operations.

As shown in Figure 11, VERiSTRONG outperforms all competitors in verifying both isolation levels. In particular, with only 16k transactions, it achieves up to a 4.4x speedup, even compared to Cobra with GPU acceleration. Interestingly, even with a stronger H800 GPU (compared to the V100 used in the original experiments [32]), VERiSTRONG still maintains the performance gap. This suggests that the primary bottleneck in verification lies not in GPU capability but in the solving stage, which is consistent with the fact that GPUs are mainly used to accelerate reachability analysis before solving. For SNAPSHOT ISOLATION, VERiSTRONG delivers up to a 4x speedup over PolySI and substantially outperforms Viper and dbcop. All these results highlight the benefits of our dedicated SMT solving.

**7.2.3 Scalability.** Verifying large histories is highly desirable for increasing confidence in a database's fulfilment of promised isolation guarantee. Following the previous experiments, we evaluate VERiSTRONG's scalability on large histories, each containing up to 100k transactions and 2 million operations with a total of 50k keys.<sup>4</sup>

As shown in Figure 12, verifying large histories is manageable for VERiSTRONG on modern hardware, e.g., SERIALIZABILITY verification completes in under 5 hours using less than 64 GB of memory. In addition, two expected trends are observed. First, both runtime and memory usage increase as the proportion of read operations decreases (from RH to BL to WH workloads). This trend is primarily due to the increased uncertainty in WW dependencies, which expands the set of potential read-from candidates and ultimately amplifies the overall uncertainty.

Second, verifying SNAPSHOT ISOLATION incurs higher overhead than SERIALIZABILITY. This is expected: while SERIALIZABILITY requires only an acyclic dependency graph, SNAPSHOT ISOLATION permits certain cycles, making the analysis inherently more complex. In practice, VERiSTRONG maintains an additional induced graph for SNAPSHOT ISOLATION, which is more expensive to construct than deriving RW edges alone as for SERIALIZABILITY. This gap is also evident in prior evaluations on UniqueValue histories: Cobra verifies a history with 10k transactions against SERIALIZABILITY in 15s [32], whereas Viper requires over 400s to verify SNAPSHOT ISOLATION for the same amount of transactions [37]. This gap becomes even more pronounced in the presence of duplicate values.

<sup>4</sup>We used a server with an AMD EPYC 7H12 64-Core processor and 1TB of memory.

Meta-4

Meta-4

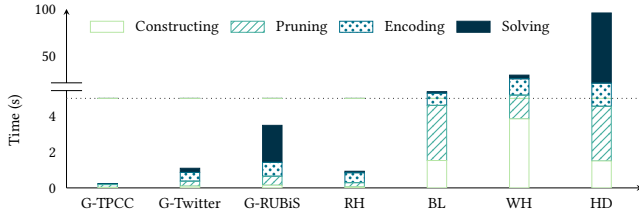


Figure 13: Breakdown of checking time across stages.

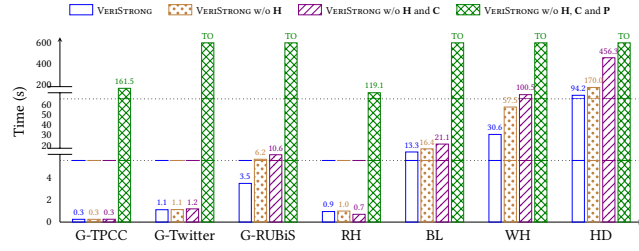


Figure 14: Ablation analysis. Timeout is set to 600s.

## 7.3 Dissecting VERISTRONG

In this section, we take a closer look at VERISTRONG, examining how its individual components and optimization strategies contribute to the overall checking efficiency.

**7.3.1 Decomposition Analysis.** We decompose VERISTRONG’s checking time into four stages: constructing, pruning, encoding, and solving. Figure 13 presents the breakdown across seven benchmarks (see Section 7.1). Overall, the constructing and pruning stages are relatively inexpensive. The G-TPCC workload is a special case where all constraints are pruned prior to encoding and solving, resulting in negligible cost for the latter stages. In most benchmarks—except for G-RUBiS and HD—the solving time remains relatively low, whereas the encoding stage dominates. This behavior is expected: our small-width cycles optimization shifts some of the burden from solving to pruning and encoding, while keeping these two stages efficient. As a result, the overall checking time is reduced.

**7.3.2 Ablation Analysis.** To assess the contributions of the major optimizations in VERISTRONG, we consider three variants: (i) VERISTRONG without the heuristic for polarity picking (**H**); (ii) VERISTRONG without **H** and without the encoding of small-width cycles (**C**); and (iii) VERISTRONG without **H**, **C**, and pruning (**P**).

Figure 14 presents our ablation analysis results. Overall, all optimizations prove effective, though their impact varies across different workloads. Optimization **H** is particularly beneficial for G-RUBiS, BL, WH, and HD, but shows little effect on G-Twitter and RH. This is because the latter two are read-heavy workloads, where cycles are rarely encountered during solving. Consequently, the potential for **H** to resolve conflicts through polarity picking is limited.

We also observe that the workloads where Optimization **C** is most effective largely overlap with those where **H** is effective. Interestingly, for RH, Optimization **C** slightly increases the overall checking time. This is expected, as the overhead stems from the enumeration of small-width cycles, which might be unnecessary in cycle-sparse workloads like RH.

Finally, we find that Optimization **P** provides significant gains, as a large number of 1-width cycles are eliminated, including WR

constraints and the associated RW ones unique to DuplicateValue histories. For instance, as noted earlier, all constraints in G-TPCC are pruned, allowing the solving phase to be bypassed entirely.

**7.3.3 Impact of Cycle Width.** Recall that in Optimization **C**, we encode 2-width cycles (see Section 6.1.2). A natural question arises: what is the performance impact of encoding cycles of larger widths? To answer this, we conduct a comparative experiment using variants of Variant (i), varying the width of encoded cycles. We plot the results in Figure 15, where  $w = 1$  corresponds to a baseline with pruning only (i.e., no cycle encoding), while  $w \geq 2$  indicates that cycles of width  $w$  are explicitly encoded. Across all six benchmarks (excluding G-TPCC, where solving is bypassed), encoding 2-width cycles ( $w = 2$ ) consistently delivers the best performance, striking a balance between pruning effectiveness and solving overhead. The only exception is RH, where solving time is already low, suggesting that in cycle-sparse workloads, the modest overhead of encoding small-width cycles has limited impact on overall performance.

**7.3.4 Impact of Polarity Picking.** To better understand how Optimization **H** improves VERISTRONG’s performance, we examine its effect on reducing the number of encountered conflicts (i.e., the number of backtracks) during solving. We conduct a comparative experiment between VERISTRONG and Variant (i) across seven benchmarks. As shown in Table 1, **H** significantly reduces the number of conflicts. This effect becomes more pronounced as the write proportion and the duplication rate of write values increase, which is consistent with the performance differences observed in Figure 14. The only two exceptions are G-TPCC and RH, both of which exhibit no conflicts. In G-TPCC, all constraints are pruned prior to solving. In RH, although not all constraints are pruned, its read-heavy nature results in conflict-free solving even without **H**.

## 7.4 Effectiveness

Beyond checking performance, an essential criterion for a verifier is its ability to accurately detect isolation anomalies. To this end, we validate VERISTRONG against an extensive set of known anomalies.

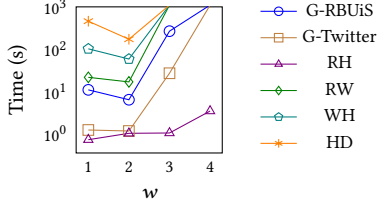
**Reproducing UniqueValue Anomalies.** VERISTRONG reproduces all known anomalies from a substantial set of 2507 UniqueValue histories: 2073 for SERIALIZABILITY and 434 for SNAPSHOT ISOLATION. These histories were originally collected by prior work [5, 14, 32] from earlier versions of widely-used databases, including CockroachDB and YugabyteDB. In addition, we report the runtime of each tool: as shown in Table 2, VERISTRONG achieves significantly lower average checking times than the state-of-the-art verifiers.

**(Re)discovering DuplicateValue Anomalies.** Motivated by recent bug reports [23, 26], we apply VERISTRONG to test both MySQL and MariaDB. VERISTRONG replicates the reported anomalies in earlier versions of both databases and, notably, rediscovers the same bug in the latest stable version of MariaDB (v 11.5.2). We reported this issue to the developers, who confirmed it as a valid bug. Note that all these anomalies lie beyond the capabilities of existing verifiers, which are limited to verifying UniqueValue histories.

## 8 Related Work

Recent years have seen significant progress in verifying database isolation guarantees. We focus on the black-box approaches.





**Figure 15: Runtime of Variant (i) under varying encoded cycle widths ( $w$ ). Time-out (600s) data points are not plotted.**

A recent line of work [14, 32, 37], including Cobra and PolySI, applies SMT solving to verify strong isolation. All these tools encode histories as polygraphs or their variants and rely on the off-the-shelf solver MonoSAT to detect cycles that represent anomalies.

VERISTRONG follows this approach but differs in three key ways. First, rather than relying solely on general-purpose SMT solvers, it incorporates workload-specific optimizations to enhance performance. Second, existing polygraph-based approaches are limited to UniqueValue histories. VERISTRONG introduces an expressive hyper-polygraph representation that also supports DuplicateValue, thereby broadening applicability. Third, VERISTRONG provides both soundness and completeness, ensuring reliable isolation verification. In contrast, existing verifiers risk false positives and false negatives, particularly in the presence of duplicate write values.

Non-solver tools [5, 18, 20, 25, 34] employ graph traversal algorithms, such as DFS or optimized variants, to verify isolation guarantees. The dbcop tool [5] applies a polynomial-time algorithm to verify SERIALIZABILITY over UniqueValue histories (with a bounded number of sessions), along with a polynomial-time reduction from verifying SNAPSHOT ISOLATION to verifying SERIALIZABILITY. However, dbcop has been reported to be less efficient than Cobra [32] and PolySI [14], which in turn are outperformed by VERISTRONG.

Elle [18], the isolation checker used in the Jepsen framework [15], infers version orders (i.e., WW dependencies) from list-append workloads. However, it relies on the UniqueValue assumption, which limits its effectiveness when applied to general workloads containing duplicate write values (see Section 9). MTC [34] exploits a class of simplified database workloads, called mini-transactions (e.g., read-modify-write transactions) to achieve quadratic-time verification under the same UniqueValue assumption. While MTC strikes a practical balance between performance and bug-finding effectiveness, its applicability is limited, as real-world workloads often diverge from such simplified transaction patterns.

Other tools, such as Plume [20] and AWDIT [25], focus on verifying weaker isolation guarantees [2, 21]. These properties are known to be verifiable in polynomial time [5] and are computationally less complex than the stronger ones we target.

## 9 Discussion and Conclusion

**Supporting List-Append Workloads.** Elle has proven highly effective for verifying strong isolation guarantees. Its key innovation lies in leveraging Jepsen’s list-append operations to efficiently infer uncertainties in WW dependencies. A list-append history consists of two types of operations: append (A), which installs a value (analogous to a write), and read (R), which returns a list of installed

**Table 1: The number of conflicts encountered during solving with or without Optimization H.**

| Benchmark | VERISTRONG | w/o H |
|-----------|------------|-------|
| G-TPCC    | 0          | 0     |
| G-Twitter | 79         | 99    |
| G-RBUIs   | 89         | 246   |
| RH        | 0          | 0     |
| BL        | 50         | 497   |
| WH        | 86         | 1427  |
| HD        | 936        | 2047  |

**Table 2: Average checking time on anomalous histories (excluding those that timed out beyond 900s).**

| Verifier   | SER    | SI     |
|------------|--------|--------|
| Cobra      | 1293ms | –      |
| PolySI     | –      | 359ms  |
| Viper      | –      | 6559ms |
| dbcop      | 551ms  | 108ms  |
| VERISTRONG | 143ms  | 18ms   |
| #histories | 2073   | 434    |

values. The order of elements in the returned list reflects the version order among the corresponding writes, thereby resolving WW ambiguities. For example, reading the list  $[2, 1]$  on key  $x$  indicates that  $A(x, 2)$  precedes  $A(x, 1)$  in the version order. In contrast, such ordering remains ambiguous in the read-write register setting.

However, Elle becomes ineffective when DuplicateValue is introduced. For instance, reading a list  $[1, 1]$  provides no information about the ordering between two append operations with the same value. This exposes a fundamental limitation of Elle in handling general histories, which remains an open challenge.

**Supporting Other Formalisms.** Our hyper-polygraphs are not restricted to Adya’s formalism; rather, they are expressive enough to support other dependency-based characterizations of isolation guarantees, such as the axiomatic framework proposed by Biswas and Enea [5]. These frameworks were introduced to capture more recent isolation levels like READ ATOMIC [3]. Our approach supports them by enabling precise modeling of both certain and uncertain dependencies, e.g., commit order [5], which, like WW, models the version order. As a demonstration, we have encoded Biswas and Enea’s framework using hyper-polygraphs; see the Appendix [6].

**Supporting Weak Isolation Levels.** While our dedicated SMT solving is highly efficient for verifying strong isolation levels, it may be less suitable for weaker ones, where SMT may be unnecessarily heavyweight [20]. In practice, users can combine VERISTRONG with existing weak isolation verifiers [20, 25] to achieve broader coverage. This integration is straightforward in the black-box setting, as all tools share a unified input format. We have realized such an integration with Plume in the IsoVista system [12].

**Limitations.** While our work lifts the strong UniqueValue assumption made by prior verifiers, it currently targets key-value databases. Extending our approach to relational models is interesting future work, potentially by building on techniques from [16, 36]. In addition, our current implementation is designed for offline verification. While it can, in principle, be applied in online settings, doing so efficiently would require addressing runtime-specific challenges, such as garbage collection of historical transactions.

**Conclusion.** We have presented a novel approach for verifying strong database isolation, centered around hyper-polygraphs—a general formalism for modeling dependency-based isolation semantics. We have focused on Adya’s theory and established sound and complete characterizations for both SERIALIZABILITY and SNAPSHOT ISOLATION. Our verifier achieves high efficiency by tailoring SMT solving to database workload characteristics, in contrast to the state-of-the-art that relies solely on general-purpose solvers.

R3-2(2)



## References

- [1] Atul Adya. 1999. *Weak Consistency: A Generalized Theory and Optimistic Implementations for Distributed Transactions*. Ph.D. Dissertation. USA.
- [2] Peter Bailis, Aaron Davidson, Alan Fekete, Ali Ghodsi, Joseph M. Hellerstein, and Ion Stoica. 2013. Highly Available Transactions: Virtues and Limitations. *Proc. VLDB Endow.* 7, 3 (nov 2013), 181–192.
- [3] Peter Bailis, Alan Fekete, Ali Ghodsi, Joseph M. Hellerstein, and Ion Stoica. 2016. Scalable Atomic Visibility with RAMP Transactions. *ACM Trans. Database Syst.* 41, 3, Article 15 (July 2016).
- [4] Sam Bayless, Noah Bayless, Holger H. Hoos, and Alan J. Hu. 2015. SAT Modulo Monotonic Theories. In *AAAI 2015*. AAAI Press, 3702–3709.
- [5] Ranadeep Biswas and Constantin Enea. 2019. On the Complexity of Checking Transactional Consistency. *Proc. ACM Program. Lang.* 3, OOPSLA, Article 165 (2019).
- [6] Zhiheng Cai, Si Liu, Hengfeng Wei, Yuxing Chen, and Anqun Pan. 2025. *Fast Verification of Strong Database Isolation*. Technical Report. <https://github.com/Czxingchen/VeriStrong/blob/main/tech-rpt.pdf>.
- [7] Andrea Cerone and Alexey Gotsman. 2018. Analysing Snapshot Isolation. *J. ACM* 65, 2, Article 11 (Jan. 2018).
- [8] Martin Davis, George Logemann, and Donald Loveland. 1962. A Machine Program for Theorem-Proving. *Commun. ACM* 5, 7 (July 1962), 394–397.
- [9] Niklas Eén and Niklas Sörensson. 2003. An Extensible SAT-solver. In *Theory and Applications of Satisfiability Testing, 6th International Conference, SAT 2003 (LNCS, Vol. 2919)*. Springer, 502–518.
- [10] Hongyu Fan, Zhihang Sun, and Fei He. 2023. Satisfiability Modulo Ordering Consistency Theory for SC, TSO, and PSO Memory Models. *ACM Trans. Program. Lang. Syst.* 45, 1, Article 6 (March 2023), 37 pages.
- [11] Shabnam Ghasemirad, Si Liu, Christoph Sprenger, Luca Multazzu, and David Basin. 2025. VerIso: Verifiable Isolation Guarantees for Database Transactions. *Proc. VLDB Endow.* 18, 5 (2025), 1362–1375.
- [12] Long Gu, Si Liu, Tiancheng Xing, Hengfeng Wei, Yuxing Chen, and David Basin. 2024. IsoVista: Black-Box Checking Database Isolation Guarantees. *Proc. VLDB Endow.* 17, 12 (Aug. 2024), 4325–4328.
- [13] Fei He, Zhihang Sun, and Hongyu Fan. 2021. Satisfiability Modulo Ordering Consistency Theory for Multi-threaded Program Verification. In *PLDI '21*. ACM, 1264–1279.
- [14] Kaile Huang, Si Liu, Zhenhe Chen, Hengfeng Wei, David A. Basin, Haixiang Li, and Anqun Pan. 2023. Efficient Black-box Checking of Snapshot Isolation in Databases. *Proc. VLDB Endow.* 16, 6 (2023), 1264–1276.
- [15] Jepsen. Accessed in May, 2025. <https://jepsen.io>.
- [16] Zu-Ming Jiang, Si Liu, Manuel Rigger, and Zhendong Su. 2023. Detecting Transactional Bugs in Database Engines via Graph-Based Oracle Construction. In *OSDI 23*. USENIX Association, 397–417.
- [17] Nick Kallen. Accessed in May, 2025. Big Data in Real Time at Twitter. <https://www.infoq.com/presentations/Big-Data-in-Real-Time-at-Twitter/>.
- [18] Kyle Kingsbury and Peter Alvaro. 2020. Elle: Inferring Isolation Anomalies from Experimental Observations. *Proc. VLDB Endow.* 14, 3 (2020), 268–280.
- [19] Donald E. Knuth. 2015. *The Art of Computer Programming, Volume 4, Fascicle 6: Satisfiability* (1st ed.). Addison-Wesley Professional.
- [20] Si Liu, Long Gu, Hengfeng Wei, and David Basin. 2024. Plume: Efficient and Complete Black-Box Checking of Weak Isolation Levels. *Proc. ACM Program. Lang.* 8, OOPSLA2, Article 302 (Oct. 2024), 29 pages. doi:10.1145/3689742
- [21] Si Liu, Luca Multazzu, Hengfeng Wei, and David A. Basin. 2024. NOC-NOC: Towards Performance-optimal Distributed Transactions. *Proc. ACM Manag. Data* 2, 1, Article 9 (March 2024), 25 pages.
- [22] Si Liu, Peter Csaba Ölveczky, Min Zhang, Qi Wang, and José Meseguer. 2019. Automatic Analysis of Consistency Properties of Distributed Transaction Systems in Maude. In *TACAS 2019 (LNCS, Vol. 11428)*. Springer, 40–57.
- [23] MariaDB-#26642. 2022. Weird SELECT view when a record is modified to the same value by two transactions. <https://jira.mariadb.org/browse/MDEV-26642>.
- [24] Anders Alnor Mathiasen, Léon Gondelman, Léon Ducruet, Amin Timany, and Lars Birkedal. 2025. Reasoning about Weak Isolation Levels in Separation Logic. *Proc. ACM Program. Lang.* ICFP (2025).
- [25] Lasse Møldrup and Andreas Pavlogiannis. 2025. AWDIT: An Optimal Weak Database Isolation Tester. *Proc. ACM Program. Lang.* 9, PLDI, Article 209 (June 2025), 25 pages.
- [26] MySQL-#100328. 2020. Inconsistent behavior with isolation levels when binlog enabled. <https://bugs.mysql.com/bug.php?id=100328>.
- [27] Robert Nieuwenhuis, Albert Oliveras, and Cesare Tinelli. 2006. Solving SAT and SAT Modulo Theories: From an abstract Davis–Putnam–Logemann–Loveland procedure to DPLL(T). *J. ACM* 53, 6 (Nov. 2006), 937–977. doi:10.1145/1217856.1217859
- [28] Christos H. Papadimitriou. 1979. The Serializability of Concurrent Database Updates. *J. ACM* 26, 4 (oct 1979), 631–653. doi:10.1145/322154.322158
- [29] David J. Pearce and Paul H. J. Kelly. 2006. A Dynamic Topological Sort Algorithm for Directed Acyclic Graphs. *ACM J. Exp. Algorithmics* 11 (2006).
- [30] Robert Robere, Antonina Kolokolova, and Vijay Ganesh. 2018. The Proof Complexity of SMT Solvers. In *Computer Aided Verification*, Hana Chockler and Georg Weissenbacher (Eds.). Springer International Publishing, Cham, 275–293.
- [31] RUBiS. Accessed in May, 2025. Auction Site for e-Commerce Technologies Benchmarking. <https://projects.ow2.org/view/rubis/>.
- [32] Cheng Tan, Changgeng Zhao, Shuai Mu, and Michael Walfish. 2020. Cobra: Making Transactional Key-Value Stores Verifiably Serializable. In *OSDI 2020*. USENIX Association, 63–80.
- [33] TPC. Accessed in October, 2023. TPC-C: On-Line Transaction Processing Benchmark. <https://www.tpc.org/tpcc/>.
- [34] Hengfeng Wei, Jiang Xiao, Na Yang, Si Liu, Zijing Yin, Yuxing Chen, and Anqun Pan. 2025. Boosting End-to-End Database Isolation Checking via Mini-Transactions. In *ICDE 2025*. IEEE Computer Society, 3998–4010.
- [35] Gerhard Weikum and Gottfried Vossen. 2002. *Transactional Information Systems: Theory, Algorithms, and the Practice of Concurrency Control and Recovery*. Morgan Kaufmann.
- [36] Rui Yang, Ziyu Cui, Wensheng Dou, Yu Gao, Jiansen Song, Xudong Xie, and Jun Wei. 2025. Detecting Isolation Anomalies in Relational DBMSs. *Proc. ACM Softw. Eng.* 2, ISSTA, Article ISSTA076 (June 2025), 23 pages.
- [37] Jian Zhang, Ye Ji, Shuai Mu, and Cheng Tan. 2023. Viper: A Fast Snapshot Isolation Checker. In *EuroSys 2023*. ACM, 654–671.

## A Verifying SNAPSHOT ISOLATION

To demonstrate the generality of our proposed techniques, this section summarizes how hyper-polygraphs can be used to characterize SNAPSHOT ISOLATION (SI), a widely used strong isolation guarantee in practice, and how our optimizations extend naturally to verifying histories against SI.

Unlike verifying SERIALIZABILITY, where detecting any cycle is sufficient to determine a violation, verifying SNAPSHOT ISOLATION is more complex, requiring identifying cycles that do not contain two adjacent RW edges [7, Theorem 4.1]. Inspired by prior work [14], we define the *induced SI graph* based on a directed labeled graph that is compatible with the hyper-polygraph of a history (as specified in Definition 3.3).

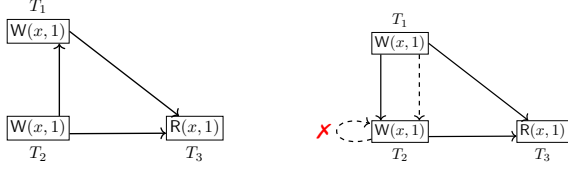
**Definition A.1.** Let  $\mathcal{G}' = (\mathcal{V}', \mathcal{E}')$  be a graph compatible with a hyper-polygraph  $\mathcal{G}$ . Let  $\mathcal{E}_T$  ( $T \in \text{Type}$ ) denote the subset of  $\mathcal{E}$  that consists of edges labeled with  $T$ . The *induced SI graph* of  $\mathcal{G}'$  is defined as

$$\mathcal{G}'_I = (\mathcal{V}', (\mathcal{E}'_{SO} \cup \mathcal{E}'_{WR} \cup \mathcal{E}'_{WW}) ; \mathcal{E}'_{RW?})^5$$

We can then characterize SNAPSHOT ISOLATION using hyper-polygraphs as follows. Illustrative examples are given below, and the proof is provided in Section B.2.

**THEOREM A.2.** *A history  $\mathcal{H}$  satisfies SNAPSHOT ISOLATION if and only if  $\mathcal{H} \models \text{INT}$  and there exists a graph compatible with the hyper-polygraph of  $\mathcal{H}$  whose induced SI graph is acyclic.*

The following example illustrates the induced SI graph, reusing the examples in Figure 3.



(a) The induced SI graph of the graph in Figure 3c. (b) The induced SI graph of the graph in Figure 3d.

**Figure 16: Induced SI graphs of compatible graphs in Figure 3.**

**Example A.3.** We reuse the history in Example 3.2 (Figure 3) to illustrate induced SI graphs. Figure 16 shows induced SI graphs of the compatible graphs in Figure 3. The edges of induced SI graph of the graph in Figure 3c (shown in Figure 16a) are the same as the compatible graph, since there is no RW edge. In contrast, the induced SI graph of the graph in Figure 3d (shown in Figure 16b) consists of two new induced edges (shown as dashed arrows):  $T_1 \rightarrow T_2$  (induced from  $T_1 \xrightarrow{WR(x)} T_3$  and  $T_3 \xrightarrow{RW(x)} T_2$ ) and  $T_2 \rightarrow T_2$  (induced from  $T_2 \xrightarrow{SO} T_3$  and  $T_3 \xrightarrow{RW(x)} T_2$ ). Note that  $T_2 \rightarrow T_2$  is actually a self-loop, indicating that the induced SI graph is cyclic.

The high-level workflow for verifying SNAPSHOT ISOLATION is aligned with that for SERIALIZABILITY, as illustrated in Figure 7. The key difference lies in the theory solver: for SNAPSHOT ISOLATION,

<sup>5</sup>Here, we slightly abuse the notation of  $;$  and  $?$ , where  $\mathcal{E}_T$  is treated as a subset of  $\mathcal{V} \times \mathcal{V}$  by selecting edges labeled  $T$  in  $\mathcal{E}$  and ignoring keys.

the verification procedure maintains the induced SI graph constructed from the known graph. Both cycle detection and conflict clause generation are performed on this induced graph. Additionally, the small-width cycle optimization is adapted to prune and encode cycles in the induced SI graph, based on the composition rule  $(SO \cup WR \cup WW) ; RW?$ . The polarity picking heuristic applies straightforwardly in this setting.

## B Proofs

### B.1 Proof of Theorem 3.5

**PROOF.** The proof proceeds in two directions.

( $\Rightarrow$ ) Suppose that  $\mathcal{H}$  satisfies SERIALIZABILITY. By Theorem 2.4,  $\mathcal{H}$  satisfies INT, and there exists WR, WW and RW relations with which  $\mathcal{H}$  can be extended to a dependency graph  $\mathcal{G}$  such that  $SO_{\mathcal{G}} \cup WR_{\mathcal{G}} \cup WW_{\mathcal{G}} \cup RW_{\mathcal{G}}$  is acyclic. Denote the hyper-polygraph of  $\mathcal{H}$  as  $\mathcal{G}' \triangleq (\mathcal{V}', \mathcal{E}', \mathcal{C}')$ . We construct an acyclic compatible graph  $\mathcal{G}'' \triangleq (\mathcal{V}'', \mathcal{E}'')$  of  $\mathcal{G}'$  based on  $\mathcal{G}$ , where  $\mathcal{V}'' = \mathcal{V}'$  corresponds to transactions of  $\mathcal{H}$  by definition, and  $\mathcal{E}'' = SO_{\mathcal{G}} \cup WR_{\mathcal{G}} \cup WW_{\mathcal{G}} \cup RW_{\mathcal{G}}$  with each edge labeled by its relation name.  $\mathcal{G}''$  is acyclic and satisfies Definition 3.3.

( $\Leftarrow$ ) Suppose that  $\mathcal{H}$  satisfies INT and there exists an acyclic compatible graph  $\mathcal{G}''$  of the hyper-polygraph  $\mathcal{G}'$  of  $\mathcal{H}$ . We construct a dependency graph  $\mathcal{G}$  based on  $\mathcal{G}''$ , where  $SO_{\mathcal{G}}, WR_{\mathcal{G}}, WW_{\mathcal{G}}$  and  $RW_{\mathcal{G}}$  consist a partition of  $\mathcal{E}''$  by their labels. For example,  $SO_{\mathcal{G}} = \bigcup_{T \xrightarrow{SO} S} (T, S)$ , and  $\forall x \in \text{Key}. WR_{\mathcal{G}}(x) = \bigcup_{T \xrightarrow{WR(x)} S} (T, S)$  (the same for WW and RW).  $\mathcal{G}$  is a valid dependency graph by Definition 2.3. Since  $\mathcal{G}''$  is acyclic,  $SO_{\mathcal{G}} \cup WR_{\mathcal{G}} \cup WW_{\mathcal{G}} \cup RW_{\mathcal{G}}$  is also acyclic.  $\mathcal{G}$  satisfies SERIALIZABILITY by Theorem 2.4.  $\square$

### B.2 Proof of Theorem A.2

The proof of the hyper-polygraph-based characterization for SNAPSHOT ISOLATION closely follows the same strategy as that of SERIALIZABILITY. Both directions of the construction – deriving a valid dependency graph from a compatible graph and vice versa – are analogous and can be reused verbatim.

**PROOF.** This proof proceeds in two directions.

( $\Rightarrow$ ) Suppose that  $\mathcal{H}$  satisfies SNAPSHOT ISOLATION. By [7, Theorem 4.1],  $\mathcal{H}$  satisfies INT, and there exists WR, WW and RW relations with which  $\mathcal{H}$  can be extended to a dependency graph  $\mathcal{G}$  such that  $(SO_{\mathcal{G}} \cup WR_{\mathcal{G}} \cup WW_{\mathcal{G}}) ; RW_{\mathcal{G}}?$  is acyclic. Denote the hyper-polygraph of  $\mathcal{H}$  as  $\mathcal{G}' \triangleq (\mathcal{V}', \mathcal{E}', \mathcal{C}')$ . We construct an acyclic compatible graph  $\mathcal{G}'' \triangleq (\mathcal{V}'', \mathcal{E}'')$  of  $\mathcal{G}'$  based on  $\mathcal{G}$ , where  $\mathcal{V}'' = \mathcal{V}'$  corresponds to transactions of  $\mathcal{H}$  by definition, and  $\mathcal{E}'' = SO_{\mathcal{G}} \cup WR_{\mathcal{G}} \cup WW_{\mathcal{G}} \cup RW_{\mathcal{G}}$  with each edge labeled by its relation name. The induced SI graph of  $\mathcal{G}''$  is acyclic, and  $\mathcal{G}''$  satisfies Definition 3.3.

( $\Leftarrow$ ) Suppose that  $\mathcal{H}$  satisfies INT and there exists a compatible graph  $\mathcal{G}''$  of the hyper-polygraph  $\mathcal{G}'$  of  $\mathcal{H}$  whose induced SI graph  $\mathcal{G}'_I$  is acyclic. We construct a dependency graph  $\mathcal{G}$  based on  $\mathcal{G}''$ , where  $SO_{\mathcal{G}}, WR_{\mathcal{G}}, WW_{\mathcal{G}}$  and  $RW_{\mathcal{G}}$  consist a partition of  $\mathcal{E}''$  by their labels. For example,  $SO_{\mathcal{G}} = \bigcup_{T \xrightarrow{SO} S} (T, S)$ , and  $\forall x \in \text{Key}. WR_{\mathcal{G}}(x) = \bigcup_{T \xrightarrow{WR(x)} S} (T, S)$  (the same for WW and RW).  $\mathcal{G}$  is

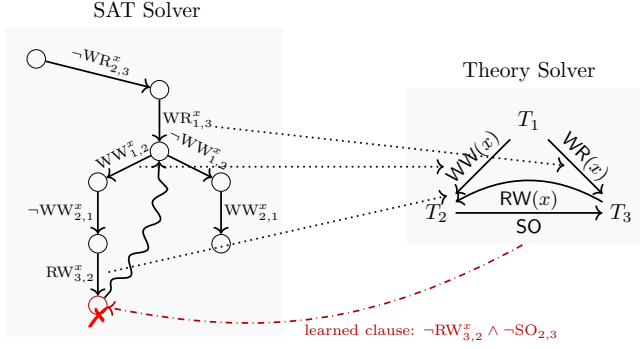


Figure 17: An illustration of how the SAT solver and the theory solver collaborate in CDCL(T) for the history  $\mathcal{H}$  in Figure 3.

a valid dependency graph by Definition 2.3. Since  $\mathcal{G}_1''$  is acyclic,  $(SO_{\mathcal{G}} \cup WR_{\mathcal{G}} \cup WW_{\mathcal{G}})$ ;  $RW_{\mathcal{G}}$  is also acyclic.  $\mathcal{G}$  satisfies SNAPSHOT ISOLATION by [7, Theorem 4.1].  $\square$

### C An Illustration of CDCL(T)

Figure 17 illustrates the interaction of the SAT solver and the theory solver when solving  $\mathcal{H}$  in Figure 3. The solving procedure is identical to that in Figure 4; see Section 4.2.

### D Other Algorithms

Algorithm 4 shows the process of constructing the hyper-polygraph  $\mathcal{G}$  of a history  $\mathcal{H}$ . It is called at Line 2 in Algorithm 1 (Section 6).

#### Algorithm 4 Construct the hyper-polygraph for a history $\mathcal{H}$

$\mathcal{G} = (\mathcal{V}, \mathcal{E}, C)$ : the known graph; initially  $(\mathcal{T}, \emptyset, (\emptyset, \emptyset))$

```

1: function CONSTRUCT( $\mathcal{H}$ )
2:   for  $T, S \in \mathcal{T}$  such that  $T \xrightarrow{SO} S$ 
3:      $\mathcal{E} \leftarrow \mathcal{E} \cup \{T \xrightarrow{SO} S\}$ 
4:   for  $S \in \mathcal{T}$  such that  $S \vdash R(x, v) \wedge |\{T \in \mathcal{T} \mid T \vdash W(x, v)\}| = 1$ 
5:      $\mathcal{E} \leftarrow \mathcal{E} \cup \{T \xrightarrow{WR(x)} S\}$ 
6:    $C^{WW} = \left\{ \{T \xrightarrow{WW(x)} S, S \xrightarrow{WW(x)} T\} \mid T \in \text{WriteTx}_x \wedge S \in \right.$ 
    $\left. \text{WriteTx}_x \wedge T \neq S \right\}$ 
7:    $C^{WR} = \left\{ \bigcup_{T_i \vdash W(x, v)} \{T_i \xrightarrow{WR(x)} S\} \mid S \vdash R(x, v) \right\}$ 

```

Algorithm 5 shows the process of pruning the hyper-polygraph  $\mathcal{G}$ . It is called at Line 3 in Algorithm 1; see Section 6.

Algorithm 6 shows the process of computing reachability of a given known graph. It is introduced in the setup part of Section 7, as an auxiliary procedure in Algorithm 5 (PRUNE) and Algorithm 2 (ENCODECYCLESOFWIDTH2) in acyclicity testing.

Algorithm 7 shows the process of encoding. It is called at Line 5 in Algorithm 1; see Section 6.

To generate conflict clauses, we introduce a boolean expression *reason* for each edge in  $G^*$ . Let  $\mathbb{F}$  denote the set of all boolean formulas. Formally, we define  $G^* \triangleq (\mathcal{V}, E^*)$ , where  $E^* \subseteq \mathcal{V} \times \mathcal{V} \times \mathbb{F}$ . We write  $T \xrightarrow{\text{reason}} S$  to denote an edge  $(T, S, \text{reason})$  in  $G^*$ , where

#### Algorithm 5 Prune

```

1: function PRUNE( $\mathcal{G}$ )
2:   repeat
3:     for  $\text{cons} \leftarrow \{\text{either} \triangleq T \xrightarrow{WW(x)} S, \text{or} \triangleq S \xrightarrow{WW(x)} T\} \in C^{WW}$ 
4:        $\text{either\_edges} \leftarrow \{\text{either}\} \cup \{S' \xrightarrow{RW(x)} S \mid T \xrightarrow{WR(x)} S' \in \mathcal{E}\}$ 
5:        $\text{or\_edges} \leftarrow \{\text{or}\} \cup \{T' \xrightarrow{RW(x)} T \mid S \xrightarrow{WR(x)} T' \in \mathcal{E}\}$ 
6:       if  $\mathcal{E} \cup \text{either\_edges}$  is cyclic
7:          $\text{cons} \leftarrow C \setminus \{\text{either}\}$ 
8:          $C^{WW} \leftarrow C^{WW} \setminus \{\text{cons}\}$ 
9:       if  $\mathcal{E} \cup \text{or\_edges}$  is cyclic
10:         $\text{cons} \leftarrow C \setminus \{\text{or}\}$ 
11:         $C^{WW} \leftarrow C^{WW} \setminus \{\text{cons}\}$ 
12:       if  $\text{cons} = \emptyset$ 
13:         return false
14:       if  $\text{cons} = \{\text{either}\}$ 
15:          $\mathcal{E} \leftarrow \mathcal{E} \cup \text{either\_edges}$ 
16:       if  $\text{cons} = \{\text{or}\}$ 
17:          $\mathcal{E} \leftarrow \mathcal{E} \cup \text{or\_edges}$ 
18:     for  $\text{cons} \in C^{WR}$ 
19:       for  $\text{wr} \leftarrow T \xrightarrow{WR(x)} S \in \text{cons}$ 
20:          $\text{wr\_edges} \leftarrow \{\text{wr}\} \cup \{S \xrightarrow{RW(x)} T' \mid T \xrightarrow{WW(x)} T'\}$ 
21:         if  $\mathcal{E} \cup \text{wr\_edges}$  is cyclic
22:            $\text{cons} \leftarrow \text{cons} \setminus \{\text{wr}\}$ 
23:            $C^{WR} \leftarrow C^{WR} \setminus \{\text{cons}\}$ 
24:         if  $\text{cons} = \emptyset$ 
25:           return false
26:         if  $|\text{cons}| = 1 \wedge |\text{cons}| = \{\text{wr} \triangleq T \xrightarrow{WR(x)} S\}$ 
27:            $\text{wr\_edges} \leftarrow \{\text{wr}\} \cup \{S \xrightarrow{RW(x)} T' \mid T \xrightarrow{WW(x)} T'\}$ 
28:            $\mathcal{E} \leftarrow \mathcal{E} \cup \text{wr\_edges}$ 
29:   until  $\mathcal{G}$  remains unchanged
30:   return true

```

#### Algorithm 6 Compute reachability of the known graph

```

1: function REACHABILITY( $\mathcal{V}, \mathcal{E}$ )
2:    $\mathcal{R} \leftarrow 0^{|\mathcal{V}| \times |\mathcal{V}|}$ 
3:    $\text{reversed\_topo\_order} \leftarrow \text{TOPOLOGICALSORT}(\mathcal{V}, \mathcal{E})$ 
4:   for  $x \in \text{reversed\_topo\_order}$ 
5:      $\mathcal{R}_{xx} \leftarrow 1$ 
6:     for  $(x, y, \_ ) \in \mathcal{E}$ 
7:        $\mathcal{R}_x \leftarrow \mathcal{R}_x \cup \mathcal{R}_y$ 

```

#### Algorithm 7 Encode

```

1: function ENCODE( $\mathcal{V}, \mathcal{E}, C$ )
2:   for  $\{T \xrightarrow{WW(x)} S, S \xrightarrow{WW(x)} T\} \in C^{WW}$   $\triangleright$  encode WW constraints
3:      $\text{Vars} \leftarrow \text{Vars} \cup \{WW_{T,S}^x, WW_{S,T}^x\}$ 
4:      $\text{Clauses} \leftarrow \text{Clauses} \cup \{WW_{T,S}^x \vee WW_{S,T}^x, \neg WW_{T,S}^x \vee \neg WW_{S,T}^x\}$ 
5:   for  $\text{cons} \in C^{WR}$   $\triangleright$  encode WR constraints
6:      $\text{Vars} \leftarrow \text{Vars} \cup \{WR_{T,S}^x \mid T \xrightarrow{WR(x)} S \in \text{cons}\}$ 
7:      $\text{Clauses} \leftarrow \text{Clauses} \cup \left\{ \bigvee_{T \xrightarrow{WR(x)} S \in \text{cons}} WR_{T,S}^x \right\}$ 
8:   ENCODECYCLESOFWIDTH2( $\mathcal{V}, \mathcal{E}$ )  $\triangleright$  Algorithm 2

```

reason  $\in \mathbb{R}$  is the reason for the edge from  $T$  to  $S$ . Edges in the initial known graph are assigned the reason true (Line 9 in Algorithm 1).

Algorithm 8 (DERIVERWEDGES) is called at Line 27 in Algorithm 1. It shows the procedure of generating *edges*, which includes the WW (or WR) edge corresponding to the assigned boolean variable and the derived RW edges. The reason of a WW or WR edge is the corresponding boolean variable. The reason of an RW edge is the conjunction of the reasons of the WW edge and the WR edge from where it is derived (see Line 4 and Line 6).

Algorithm 9 (GENCONFLICTCLAUSE) is called at Line 29 in Algorithm 1. Given a detected cycle, the conflict clause is the negation of the conjunction of the reasons of the edges involved in the cycle (see Line 3).

---

#### Algorithm 8 Derive RW edges

---

```

1: function DERIVERWEDGES( $\mathcal{M}$ )
2:    $edges \leftarrow \emptyset$ 
3:   for  $WW_{T,T'}^x \in \mathcal{M}$   $\triangleright$  WW variables that are assigned true
4:      $edges \leftarrow edges \cup \{T \xrightarrow{WW_{T,T'}^x} T' \} \cup \{S \xrightarrow{WW_{T,T'}^x} T' \mid T \xrightarrow{WR(x)}$ 
        $S \in \mathcal{E}\} \cup \{S \xrightarrow{WW_{T,T'}^x \wedge WR_{T,S}^x} T' \mid T \xrightarrow{WR_{T,S}^x} S \in E^*\}$ 
5:   for  $WR_{T,S}^x \in \mathcal{M}$   $\triangleright$  WR variables that are assigned true
6:      $edges \leftarrow edges \cup \{T \xrightarrow{WR_{T,S}^x} S\} \cup \{S \xrightarrow{WR_{T,S}^x} T' \mid T \xrightarrow{WW(x)}$ 
        $T' \in \mathcal{E}\} \cup \{S \xrightarrow{WW_{T,T'}^x \wedge WR_{T,S}^x} T' \mid T \xrightarrow{WW_{T,T'}^x} T' \in E^*\}$ 
7:   return  $edges$ 
```

---



---

#### Algorithm 9 Generate conflict clause

---

```

1: function GENCONFLICTCLAUSE( $E^*, edges$ )
2:    $cycle \triangleq \{T_1 \xrightarrow{reason_1} T_2, T_2 \xrightarrow{reason_2} T_3, \dots, T_n \xrightarrow{reason_n} T_1\} \leftarrow \text{CY-}$ 
      $\text{CLE}(E^* \cup edges)$ 
3:    $conflict\_clause \leftarrow \bigvee_{T_i \xrightarrow{reason_i} T_{i+1} \in cycle} (\neg reason_i)$ 
4:   return  $conflict\_clause$ 
```

---

## E Correctness of VERISTRONG

In this section, we establish the correctness of VERISTRONG with respect to SERIALIZABILITY through its individual optimizations.

### E.1 Small-Width Cycle Preprocessing

**Eliminating 1-width Cycles.** We formalize the procedure of eliminating 1-width cycles (i.e., pruning) using six transition rules on states of form  $\langle \mathcal{E}, C^{WW}, C^{WR} \rangle$ , as shown in Figure 18.

Pruning starts from the state  $\langle \mathcal{E}, C^{WW}, C^{WR} \rangle$ , where  $\mathcal{E}, C^{WW}$  and  $C^{WR}$  are components of the constructed hyper-polygraph  $\mathcal{G} = (\mathcal{V}, \mathcal{E}, (C^{WW}, C^{WR}))$  (see Algorithm 4). The transition rules are applied iteratively. When multiple rules are applicable, they are applied non-deterministically. Pruning terminates once no rule can be applied. Let  $\langle \mathcal{E}_p, C_p^{WW}, C_p^{WR} \rangle$  be the final state, the pruned hyper-polygraph  $\mathcal{G}_p$  is  $(\mathcal{V}, \mathcal{E}_p, (C_p^{WW}, C_p^{WR}))$ .

In Figure 18, we use  $\hookrightarrow$  to denote state transitions, distinguishing it from  $\rightarrow$  (edge) and  $\leadsto$  (reachability). Pruning relies on two main transitions ELIM and INTRO. ELIM removes an infeasible choice that

create cycles in the known graph. When a constraint has been reduced to contain a single edge, INTRO resolves it by removing it from  $C$  and adding the remaining edge to the known graph. The prefix WW- and WR- of the rule names indicate the application of them to WW and WR edges, respectively. Pruning may also terminate when encountering a violation of the isolation level, captured by the rules NO-CHOICE and CYCLE. NO-CHOICE applies when every edge would create a cycle, while CYCLE applies when the known graph itself is cyclic, implying that all compatible graphs are cyclic. Both cases yield a violation of SERIALIZABILITY, represented by the special terminal state  $S_\#$ .

Let  $n$  be the number of applied transition rules and  $s_i$  the  $i$ -th state, the pruning process can be represented as a transition chain:

$$s_0 \hookrightarrow s_1 \hookrightarrow s_2 \hookrightarrow \dots \hookrightarrow s_n,$$

where each  $s_i$  corresponds to a hyper-polygraph. Specially,  $s_0 \triangleq \langle \mathcal{E}, C^{WW}, C^{WR} \rangle$  corresponds to  $\mathcal{G}$  and  $s_n \triangleq \langle \mathcal{E}_p, C_p^{WW}, C_p^{WR} \rangle$  corresponds to  $\mathcal{G}_p$  ( $s_n$  may be  $S_\#$ ). We write  $\langle \mathcal{E}, C^{WW}, C^{WR} \rangle \hookrightarrow^* \langle \mathcal{E}_p, C_p^{WW}, C_p^{WR} \rangle$ . If  $\mathcal{H} \models \text{SER}$ , we say that the hyper-polygraph  $\mathcal{G}$  of  $\mathcal{H}$  is *SER-acyclic*.

**THEOREM E.1.** *Pruning is correct, i.e.,*

- (1) If  $\langle \mathcal{E}, C^{WW}, C^{WR} \rangle \hookrightarrow^* \langle \mathcal{E}_p, C_p^{WW}, C_p^{WR} \rangle$ , then  $\mathcal{G}$  is SER-acyclic if and only if  $\mathcal{G}_p$  is SER-acyclic.
- (2) If  $\langle \mathcal{E}, C^{WW}, C^{WR} \rangle \hookrightarrow^* S_\#$ , then  $\mathcal{G}$  is not SER-acyclic;

**PROOF OF CONDITION (1).** We first prove the condition (1) of Theorem E.1 by induction on the number of transitions. In this case, neither rule NO-CHOICE nor CYCLE is applied. Let  $s_i$  correspond to  $\mathcal{G}_i$  for  $0 \leq i \leq n$ ; in particular,  $\mathcal{G}_0 = \mathcal{G}$  and  $\mathcal{G}_n = \mathcal{G}_p$ .

- **Base Case.**  $s_0$  corresponds to  $\mathcal{G}$ , which directly satisfies condition (1).
- **Inductive Step.** Fix  $0 \leq k < n$  and assume  $\mathcal{G}_k$  is SER-acyclic if and only if  $\mathcal{G}$  is SER-acyclic, we show that  $\mathcal{G}_{k+1}$  is SER-acyclic if and only if  $\mathcal{G}_k$  is SER-acyclic. Let  $s_k \hookrightarrow s_{k+1}$  be obtained by one of the following rules:

- **WW-ELIM or WR-ELIM.** The removed edge (of type WW or WR) cannot be present in any acyclic compatible graph of  $\mathcal{G}_k$ ; otherwise, it creates a cycle in the known graph. Since the rest of the structure from  $\mathcal{G}_k$  to  $\mathcal{G}_{k+1}$  remains unchanged, the sets of acyclic compatible graphs of  $\mathcal{G}_k$  and  $\mathcal{G}_{k+1}$  are the same. Hence  $\mathcal{G}_{k+1}$  is SER-acyclic iff  $\mathcal{G}_k$  is SER-acyclic.
- **WW-INTRO or WR-INTRO.** By Definition 3.3, when a constraint is reduced to contain a single edge, the remaining edge  $e$  (and the RW edges derived from the known graph) must belong to every compatible graph of  $\mathcal{G}_k$ . Adding these edges to  $\mathcal{E}$  merely makes explicit what is already required in all compatible graphs; no other part of the structure changes. Therefore the sets of acyclic compatible graphs of  $\mathcal{G}_k$  and  $\mathcal{G}_{k+1}$  are the same, and again  $\mathcal{G}_{k+1}$  is SER-acyclic iff  $\mathcal{G}_k$  is SER-acyclic.

Thus, by induction on  $k$ ,  $\mathcal{G}_n = \mathcal{G}_p$  is SER-acyclic iff  $\mathcal{G}$  is SER-acyclic.  $\square$

Meta-1



$$\begin{array}{c}
\frac{\emptyset \in C^{WW} \cup C^{WR}}{\langle \mathcal{E}, C^{WW}, C^{WR} \rangle \hookrightarrow S_{\#}} \text{ No-CHOICE} \qquad \frac{\mathcal{E} \text{ is cyclic}}{\langle \mathcal{E}, C^{WW}, C^{WR} \rangle \hookrightarrow S_{\#}} \text{ CYCLE} \\
\\
\frac{\begin{array}{l} cons \in C^{WW} \quad T \xrightarrow{WW(x)} T' \in cons \quad cons' := cons \setminus \{T \xrightarrow{WW(x)} T'\} \\ \mathcal{E}^{WW} := \{T \xrightarrow{WW(x)} T'\} \cup \{S \xrightarrow{RW(x)} T' \mid T \xrightarrow{WR(x)} S \in \mathcal{E}\} \quad \mathcal{E} \cup \mathcal{E}^{WW} \text{ is cyclic} \end{array}}{\langle \mathcal{E}, C^{WW}, C^{WR} \rangle \hookrightarrow \langle \mathcal{E}, (C^{WW} \setminus \{cons\}) \cup \{cons'\}, C^{WR} \rangle} \text{ WW-ELIM} \\
\\
\frac{\begin{array}{l} cons \in C^{WR} \quad T \xrightarrow{WR(x)} S \in cons \quad cons' := cons \setminus \{T \xrightarrow{WR(x)} S\} \\ \mathcal{E}^{WR} := \{T \xrightarrow{WR(x)} S\} \cup \{S \xrightarrow{RW(x)} T' \mid T \xrightarrow{WW(x)} T' \in \mathcal{E}\} \quad \mathcal{E} \cup \mathcal{E}^{WR} \text{ is cyclic} \end{array}}{\langle \mathcal{E}, C^{WW}, C^{WR} \rangle \hookrightarrow \langle \mathcal{E}, C^{WW}, (C^{WR} \setminus \{cons\}) \cup \{cons'\} \rangle} \text{ WR-ELIM} \\
\\
\frac{\begin{array}{l} \{T \xrightarrow{WW(x)} T'\} \in C^{WW} \quad \mathcal{E}^{WW} := \{T \xrightarrow{WW(x)} T'\} \cup \{S \xrightarrow{RW(x)} T' \mid T \xrightarrow{WR(x)} S \in \mathcal{E}\} \end{array}}{\langle \mathcal{E}, C^{WW}, C^{WR} \rangle \hookrightarrow \langle \mathcal{E} \cup \mathcal{E}^{WW}, C^{WW} \setminus \{T \xrightarrow{WW(x)} T'\}, C^{WR} \rangle} \text{ WW-INTRO} \\
\\
\frac{\begin{array}{l} \{T \xrightarrow{WR(x)} S\} \in C^{WR} \quad \mathcal{E}^{WR} := \{T \xrightarrow{WR(x)} S\} \cup \{S \xrightarrow{RW(x)} T' \mid T \xrightarrow{WW(x)} T' \in \mathcal{E}\} \end{array}}{\langle \mathcal{E}, C^{WW}, C^{WR} \rangle \hookrightarrow \langle \mathcal{E} \cup \mathcal{E}^{WR}, C^{WW}, C^{WR} \setminus \{T \xrightarrow{WR(x)} S\} \rangle} \text{ WR-INTRO}
\end{array}$$

Figure 18: Rules for eliminating 1-width cycles.

PROOF OF CONDITION (2). As  $S_{\#}$  has no outgoing transitions, it is sufficient to show if either the rule No-CHOICE or CYCLE is applied on the last transition  $s_{n-1} \hookrightarrow s_n$ , then  $\mathcal{G}_{n-1}$  is not SER-acyclic.

- **No-CHOICE.** By Definition 3.3, if a constraint  $cons$  in  $\mathcal{G}_{n-1}$  becomes empty, no compatible graph of  $\mathcal{G}_{n-1}$  exists, since  $|\mathcal{E} \cap cons| = 1$  cannot be satisfied. Hence  $\mathcal{G}_{n-1}$  is not SER-acyclic.
- **CYCLE.** By Definition 3.3, any compatible graph of  $\mathcal{G}_{n-1}$  must include the cycle already present in the known graph. Thus,  $\mathcal{G}_{n-1}$  is not SER-acyclic.

Hence, in both cases  $\mathcal{G}_{n-1}$  is not SER-acyclic. By the equivalence already established between  $\mathcal{G}_{n-1}$  and  $\mathcal{G}$ , it follows that  $\mathcal{G}$  is not SER-acyclic.  $\square$

**Encoding 2-width Cycles.** Let  $\mathcal{G}_p$  be the hyper-polygraph after pruning and let  $\Phi_{\mathcal{G}_p}$  denote its encoding (see Section 4.1). In the encoding of 2-width cycles,  $\Phi_{\mathcal{G}_p}$  is extended to the final formula  $\mathcal{F}$  by conjoining the encoding of all identified 2-width cycles. Let  $n$  be the number of such cycles and  $\rho_i (1 \leq i \leq n)$  denote the encoding of the  $i$ -th cycle. Formally,

$$\mathcal{F} = \Phi_{\mathcal{G}_p} \wedge \bigwedge_{1 \leq i \leq n} \rho_i, \text{ where } \rho_i = \neg v_{i,1} \vee \neg v_{i,2} \text{ (see Algorithm 2).}$$

**THEOREM E.2.** *Given for the same known graph in the theory solver,  $\Phi_{\mathcal{G}_p}$  and  $\mathcal{F}$  are equisatisfiable, i.e.,  $\Phi_{\mathcal{G}_p}$  is satisfiable if and only if  $\mathcal{F}$  is satisfiable.*

**PROOF.** This proof proceeds by induction on the number  $k$  of 2-width cycles. Let  $\mathcal{F}_k = \Phi_{\mathcal{G}_p} \wedge \bigwedge_{1 \leq i \leq k} \rho_i$ , so that  $\mathcal{F}_0 = \Phi_{\mathcal{G}_p}$ .

- **Base Case.**  $\mathcal{F}_0 = \Phi_{\mathcal{G}_p}$  directly implies the equisatisfiability.
- **Inductive Step.** Assume  $\mathcal{F}_k$  is equisatisfiable with  $\Phi_{\mathcal{G}_p}$  for some  $0 \leq k < n$ . We show that  $\mathcal{F}_{k+1} = \mathcal{F}_k \wedge \rho_{k+1}$  is equisatisfiable with  $\mathcal{F}_k$ . By construction,  $\rho_{k+1} = \neg v_{k+1,1} \vee \neg v_{k+1,2}$  encodes the fact that the two edges associated with  $v_{k+1,1}$  and  $v_{k+1,2}$  cannot simultaneously exist, otherwise they would form a 2-width cycle in the known graph (see Section 6.1.2). Therefore every satisfying model of  $\mathcal{F}_k$  satisfies  $\rho_{k+1}$ . Hence  $\mathcal{F}_k$  is equisatisfiable with  $\mathcal{F}_k \wedge \rho_{k+1}$ , and by the definition of  $\mathcal{F}_{k+1}$  we obtain that  $\mathcal{F}_{k+1}$  is equisatisfiable with  $\mathcal{F}_k$ .  $\square$

## E.2 Order-Guided Polarity Picking

**THEOREM E.3.** *The polarity picking heuristic described in Section 6.2 does not affect the correctness of the CDCL(T) algorithm.*

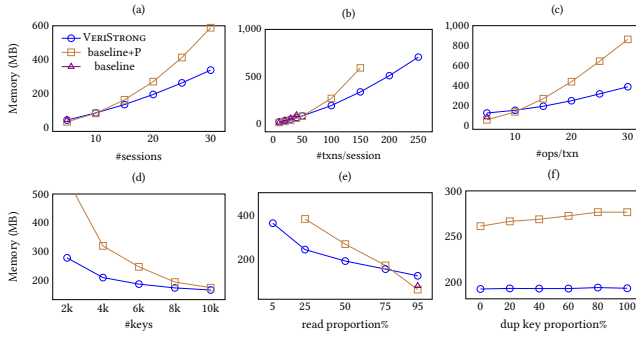
**PROOF SKETCH.** Let CDCL(T) denote the CDCL(T) algorithm without the polarity picking heuristic (H), and let C<sub>DC</sub>L(T)<sup>H</sup> denote the algorithm with the heuristic H applied. The heuristic H neither introduces new branches nor eliminates existing ones in the search. Consequently, CDCL(T) and C<sub>DC</sub>L(T)<sup>H</sup> explore the same search space, possibly in different orders. Hence, the correctness of H follows directly from the correctness of CDCL(T) [27].  $\square$

## F Memory Usage Comparison

Figure 19 shows the peak memory usage of VERISTRONG and the baseline on various general workloads, complementing Figure 10; see Section 7.2.1.

**Table 3: Notations**

| Category            | Notation                                                                                                                      | Description                                |
|---------------------|-------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------|
| KV Store            | Key                                                                                                                           | set of keys                                |
|                     | Val                                                                                                                           | set of values                              |
|                     | Op                                                                                                                            | set of operations                          |
| Dependency          | SO, WR, WW, RW                                                                                                                | dependency relations/edges                 |
| History             | $\mathcal{H} = (\mathcal{T}, \text{SO})$                                                                                      | history                                    |
| Hyper-polygraph     | $\mathcal{G} = (\mathcal{V}, \mathcal{E}, C)$                                                                                 | hyper-polygraph                            |
|                     | $C = (C^{\text{WW}}, C^{\text{WR}})$                                                                                          | constraint                                 |
|                     | $\mathcal{G}' = (\mathcal{V}', \mathcal{E}')$                                                                                 | compatible graph                           |
|                     | $T \xrightarrow{\text{SO}} S, T \xrightarrow{\text{WW}(x)} S, T \xrightarrow{\text{WR}(x)} S, T \xrightarrow{\text{RW}(x)} S$ | edges                                      |
| Propositional Logic | $v$                                                                                                                           | variable                                   |
|                     | $l = v \text{ or } \neg v$                                                                                                    | literal                                    |
|                     | $C = l_1 \vee l_2 \vee \dots \vee l_n$                                                                                        | clause                                     |
|                     | $\mathcal{F} = C_1 \wedge C_2 \wedge \dots \wedge C_m$                                                                        | formula                                    |
|                     | $\mathcal{M}$                                                                                                                 | (partial) model                            |
| SMT                 | $\text{SO}_{i,j}, \text{WW}_{i,j}^x, \text{WR}_{i,j}^x, \text{RW}_{i,j}^x$                                                    | variables corresponding to different edges |
|                     | $\Phi_{\mathcal{G}} = \Phi_{\text{KG}} \wedge \Phi_{C^{\text{WW}}} \wedge \Phi_{C^{\text{WR}}} \wedge \Phi_{\text{RW}}$       | encodings                                  |
|                     | $\text{Vars}$                                                                                                                 | set of boolean variables                   |
|                     | $\text{Clauses}$                                                                                                              | set of clauses                             |
|                     | $G^* = (\mathcal{V}, E^*)$                                                                                                    | the graph maintained in the theory solver  |



**Figure 19: Peak memory usage comparison. Timeout data points are not plotted.**

## G Characterizing Biswas et al.’s Theory

In the theory of Biswas et al. [5], a history satisfies a given isolation level if there exists a strict total order CO (called *commit order*) on its transactions that extends both the WR and SO relations and satisfies certain additional properties. In this section, we show how to characterize isolation levels under this formalism using hyper-polygraphs, taking SERIALIZABILITY as a representative example.

SERIALIZABILITY requires that for any transaction  $T_1$  writing to a key  $x$  that is read by a transaction  $T_3$ , every commit predecessor of  $T_3$  that also writes to  $x$  must precede  $T_1$  in commit order. That is,

**THEOREM G.1.** For a history  $\mathcal{H} = (\mathcal{T}, \text{SO})$ ,

$$\mathcal{H} \models \text{SER} \iff \mathcal{H} \models \text{INT} \wedge \exists \text{CO}. \forall x \in \text{Key}. \forall T_1, T_2, T_3 \in \mathcal{T}.$$

$$T_1 \neq T_2 \wedge T_1 \xrightarrow{\text{WR}(x)} T_3 \wedge T_2 \vdash \text{W}(x, \_) \wedge (T_2, T_3) \in \text{CO} \implies (T_2, T_1) \in \text{CO}.$$

The definition of hyper-polygraph under Biswas et al.’s formalism is a straightforward adaptation of Definition 3.1, with the constraint set  $C^{\text{WW}}$  replaced by  $C^{\text{CO}}$ .

**Definition G.2.** A hyper-polygraph  $\mathcal{G} = (\mathcal{V}, \mathcal{E}, C)$  for a history  $\mathcal{H} = (\mathcal{T}, \text{SO})$  is a directed labeled graph  $(\mathcal{V}, \mathcal{E})$ , referred to as the known graph, together with a pair of constraint sets  $C = (C^{\text{CO}}, C^{\text{WR}})$ , where

- $\mathcal{V}$  is the set of nodes, corresponding to the transactions in  $\mathcal{H}$ ;
- $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V} \times \text{Type} \times \text{Key}$  is a set of edges, where each edge is labeled with a dependency type from  $\text{Type} = \{\text{SO}, \text{WR}, \text{CO}\}$  and a key from  $\text{Key}$ ;
- $C^{\text{CO}}$  is a constraint set over uncertain version orders, defined as  $C^{\text{CO}} = \{T \xrightarrow{\text{CO}} S, S \xrightarrow{\text{CO}} T \mid T, S \in \mathcal{T} \wedge T \neq S\}$ ; and
- $C^{\text{WR}}$  is a constraint set over uncertain read-from mappings, defined as  $C^{\text{WR}} = \left\{ \bigcup_{T_i \vdash \text{W}(x, v)} \{T_i \xrightarrow{\text{WR}(x)} S\} \mid S \vdash \text{R}(x, v) \right\}$ .

A compatible graph of the hyper-polygraph resolves the uncertainty in CO and WR, while enforcing the CO-required properties.

**Definition G.3.** A directed labeled graph  $\mathcal{G}' = (\mathcal{V}', \mathcal{E}')$  is said to be *SER-compatible* with a hyper-polygraph  $\mathcal{G} = (\mathcal{V}, \mathcal{E}, C)$  if

- $\mathcal{V}' = \mathcal{V}$ ;
- $\mathcal{E}' \supseteq \mathcal{E}$  such that  $\forall T_1, T_2, T_3 \in \mathcal{V}. T_1 \neq T_2 \wedge T_1 \xrightarrow{\text{WR}(x)} T_3 \wedge T_2 \in \text{WriteTx}_x \wedge T_2 \xrightarrow{\text{CO}} T_3 \implies T_2 \xrightarrow{\text{CO}} T_1$ ;
- $\forall C \in C^{\text{CO}}. |\mathcal{E}' \cap C| = 1$ ; and
- $\forall C \in C^{\text{WR}}. |\mathcal{E}' \cap C| = 1$ .

Consequently, the hyper-polygraph-based characterization of SERIALIZABILITY in Biswas et al.’s formalism follows directly from Theorem G.1.

**THEOREM G.4.** A history  $\mathcal{H}$  satisfies SERIALIZABILITY if and only if  $\mathcal{H} \models \text{INT}$  and there exists an acyclic graph that is SER-compatible with the hyper-polygraph of  $\mathcal{H}$ .

Similarly, other isolation guarantees defined in Biswas et al.’s framework can also be characterized using hyper-polygraphs. Note that READ COMMITTED requires a finer-grained modeling of the program order (po).

## H Notations

Table 3 provides a summary of the notations used in this paper.

R3-1